



cBank App

PROJET DE FIN DE MODULE EN **LANGAGE C**

“GESTION DES CLIENTS ET DES COMPTES BANCAIRES”

Département de Mathématiques et Informatique

Réalisé par

Mohammed MOUSTAFID

Responsable du module

Pr. Sanaa EL FILALI

Encadré par

Mlle. Hanane ABOURIFA



Remerciement:

Nous souhaitons adresser nos sincères remerciements à Madame EL FILALI, professeure du module Langage C, pour la clarté de ses explications, la qualité de son enseignement et l'intérêt qu'elle a su susciter pour ce langage. Grâce à ses conseils et à son accompagnement pédagogique, nous avons pu acquérir des bases solides et les mettre en pratique efficacement dans la réalisation de ce projet.

Nous tenons également à remercier chaleureusement Madame ABOURIFA, encadrante de ce projet, pour sa disponibilité, son écoute et son soutien tout au long du travail. Ses remarques constructives, ses orientations précieuses et son suivi attentif nous ont permis d'améliorer continuellement notre projet et de surmonter les différentes difficultés rencontrées.

Qu'elles trouvent ici l'expression de notre profonde reconnaissance pour leur engagement, leur bienveillance et leur contribution déterminante à notre formation.

Résumé :

Ce projet s'intéresse aux limites de la gestion manuelle des opérations bancaires, souvent source d'erreurs, de manque de traçabilité et de difficultés d'accès rapide aux informations. Face à ces contraintes, il devient nécessaire de disposer d'un système informatique capable d'organiser et de sécuriser les données relatives aux clients, aux comptes et aux transactions bancaires.

L'objectif de ce travail est de concevoir et de réaliser une application de gestion bancaire permettant aux administrateurs et aux clients d'effectuer les principales opérations bancaires de manière simple et fiable. L'application vise à faciliter la gestion des comptes, l'exécution des retraits et des virements, ainsi que la consultation de l'historique des transactions, tout en assurant un contrôle d'accès de base.

Le projet intègre plusieurs fonctionnalités essentielles, notamment la gestion des clients, l'ouverture et la fermeture des comptes bancaires, la réalisation des opérations financières et la consultation des transactions. Il a été développé en langage C, en utilisant des structures de données adaptées et la gestion des fichiers afin d'assurer la conservation des informations et la continuité du système.

Les résultats obtenus montrent que l'application répond aux objectifs fixés et fonctionne de manière fiable. Les différents tests réalisés ont permis de valider la cohérence des traitements, la gestion correcte des erreurs et la persistance des données, offrant ainsi une solution simple, fonctionnelle et adaptée aux besoins de base d'un système de gestion bancaire.

Liste des tableaux :

tableau 1: Besoins Fonctionnels	14
tableau 2 :Besoins Non-Fonctionne	15
tableau 3: Les limites du projet	17
tableau 4:Analyse des Flux de Données (Entrées/Sorties)	22
tableau 5:structure Client	32
tableau 6:structure Compte	34
tableau 7:Les Fonctions ,explication et description	39
tableau 8 : Organisation des fichiers .c et .h	50
tableau 9:Environnement de Travail	54
tableau 10:Objectifs Atteints ou Non	72
tableau 11:Améliorations Possibles	73

Liste des figures :

figure 1:diagramme de cas d'utilisation	23
figure 2:diagramme de sequence processus de connexion	24
figure3::diagramme de sequence la logique de sécurité lors d'un retrait.	25
figure4::diagramme de sequence Le scénario de virement	26
figure5:algorithme d authentification	26
figure6:Algorithme de Retrait	28
figure7:Algorithme de Recherche Client	29
figure8:Les Fonctions:	30
figure9:structure Client	32
figure10:structure Compte	33
figure11:Stockage des donnees	35
figure12: Structure des fichiers de projet	36
figure13:Navigation avec Flèches (Menu Principal)	52
figure14:Validation avec Boucle Erreur/Rechargement)	53
figure15:Persistence des Transactions	53
figure16:Interface principale d'authentification et de connexion	55
figure17: Menu Principal Administrateur avec navigation intuitive	55
figure18: Menu du sous-système de gestion des clients	56
figure19:Formulaire interactif de création d'un nouveau client	56
figure20:Interface de modification granulaire des données client	57
figure21:Procédure sécurisée de suppression d'un client	58
figure22:Module de recherche multicritère de client Explication	59
figure23: Affichage tabulaire de la liste complète des clients	59
figure24:Interface de réinitialisation administrative du code PIN	60
figure25:Menu centralisé de gestion des comptes bancaires	60
figure26:Processus de création et provisionnement d'un compte	61
figure27:Vue consolidée du parc de comptes bancaires Explication	62
figure28:Procédure sécurisée de clôture d'un compte bancaire	62
figure29:Menu de sélection des opérations financières	63
figure30:Séquence interactive et sécurisée de retrait d'espèces	64
figure31:Workflow sécurisé de virement inter-comptes	65
figure32:Tableau de bord de l'espace client	66
figure33:Interface de consultation du portefeuille personnel	66
figure34: Processus de retrait d'espèces depuis l'espace client	67
figure35:Exécution sécurisée d'un virement bancaire par le client	68
figure36:Module de consultation et de filtrage de l'historique des transactions	69

Table des matières :

Remerciement	4
Résumé	5
Liste des tableaux	6
Liste des figures	7
Table des matières	8
Introduction générale	10
chapitre 1: Contexte général	11
Introduction:	13
1.1 Description du problème réel :	13
1.2 Les besoins:	14
1.2.1 Besoins Fonctionnels:	14
1.2.2 Besoins Non-Fonctionnels	15
1.3 Les objectifs fonctionnels:	15
1.4 Les limites du projet:	16
Conclusion:	17
chapitre 2: Conception	19
introduction	21
2.1 Analyse détaillée des besoins	21
2.1.1 Identification des Acteurs:	21
2.1.2 Spécification des Règles de Gestion (RG)	21
2.1.3 Analyse des Flux de Données (Entrées/Sorties)	22
2.1.4 Dictionnaire de Données	22
2.2 Les Diagrammes (UML)	23
2.2.1 diagramme de cas d'utilisation:	23
2.2.2 les diagrammes de sequence	24
2.3 Algorithmes Principaux:	26
2.3.1 algorithme d authentification	26
2.3.2 Algorithme de Retrait:	28
2.3.3 Algorithme de Recherche Client	29
2.4 Structure du programme:	30
2.4.1 Les Fonctions:	30
2.4.2 structures et Leurs Rôles:	32
2.4.2.1 structure Client	32
2.4.2.2 structure Compte	33
2.4.3 Les fichiers:	35
2.4.3.1 Stockage des données	35
2.4.3.2 Structure des fichiers de projet:	36
Conclusion	36

Table des matières :

chapitre 3:Réalisation	37
introduction:	39
3.1. Implémentation en C:	39
3.1.1 Les Fonctions ,explication et description:	39
3.1.2 Organisation des fichiers .c et .h:	50
3.1.3 Focus sur les fonctionnalités clés (Extraits de code)	52
3.2 Environnement de Travail	54
3.3 Résultats:	54
3.3.1 admin:	55
3.3.2 client:	66
Conclusion	69
Conclusion générale	70
Bilan du Projet	72
Objectifs Atteints ou Non	72
Difficultés Rencontrées	72
Améliorations Possibles	73
Ouvertures Future	73

Introduction générale:

Dans un contexte marqué par la transformation numérique des services financiers, les institutions bancaires sont de plus en plus amenées à moderniser leurs méthodes de gestion afin d'assurer une meilleure organisation des données, une plus grande fiabilité des opérations et une sécurité accrue des informations sensibles. La gestion manuelle des clients, des comptes et des transactions devient rapidement insuffisante face à l'augmentation du volume des données et aux exigences croissantes de rapidité et de précision.

C'est dans cette optique que s'inscrit le présent projet, qui consiste en la conception et la réalisation d'une application de gestion bancaire en langage C. Ce projet vise à proposer une solution informatique simple et structurée permettant de gérer les clients, les comptes bancaires et les opérations financières courantes telles que les retraits et les virements. Il prend également en compte les notions essentielles de sécurité à travers un mécanisme d'authentification de base, garantissant un accès contrôlé aux différentes fonctionnalités du système.

L'objectif principal de ce travail est de développer une application fonctionnelle, fiable et ergonomique, capable de répondre aux besoins fondamentaux d'un système bancaire. Le projet met l'accent sur la clarté de l'interface utilisateur, la persistance des données et la séparation des rôles entre l'administrateur et le client, afin d'assurer une utilisation cohérente et sécurisée du système.

Ce rapport est structuré de manière progressive afin de guider le lecteur à travers les différentes étapes du projet. Il débute par la présentation du contexte général et l'analyse des besoins, se poursuit par la conception du système et le choix des structures de données, puis détaille la phase de réalisation et de tests. Enfin, il se conclut par une évaluation des résultats obtenus, une analyse des limites du projet et la présentation des perspectives d'amélioration futures.

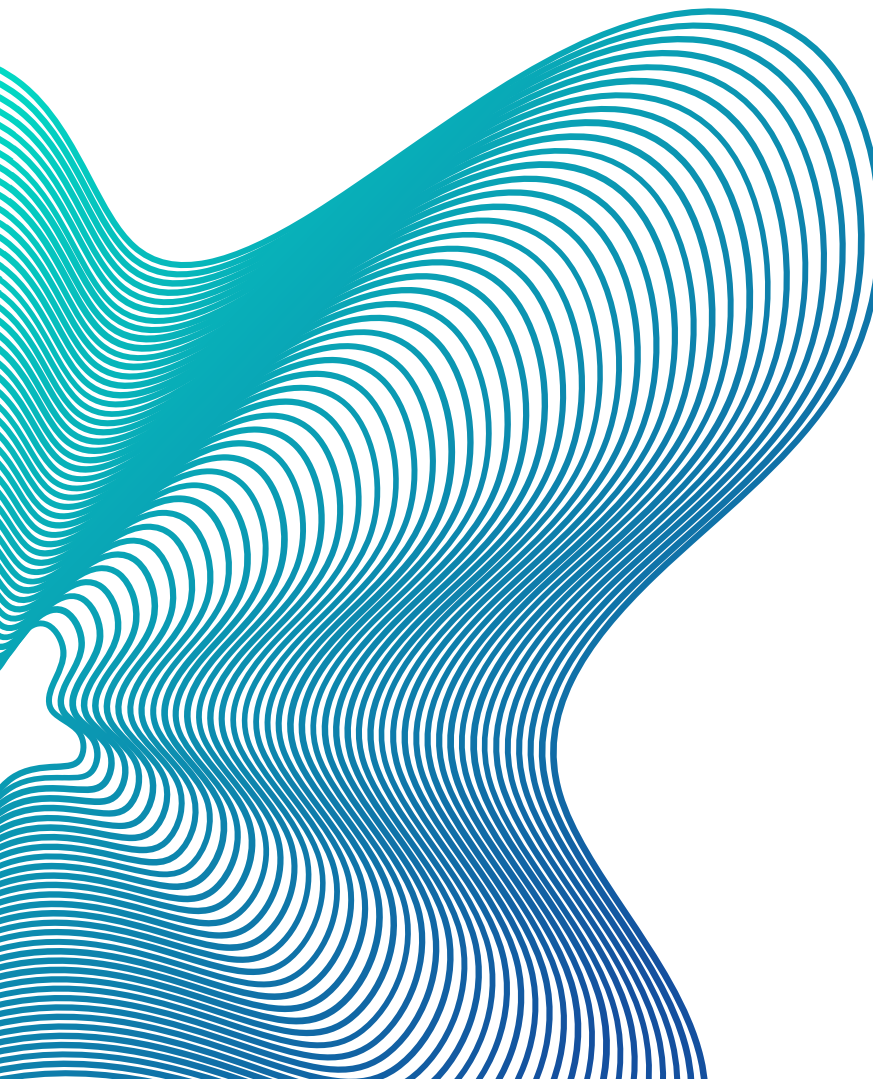
CHAPITRE 1:

CONTEXTE GÉNÉRAL



CHAPITRE 1:

CONTEXTE GÉNÉRAL



Introduction:

Avant de se lancer dans le développement technique de la solution, il est essentiel de comprendre l'environnement dans lequel le projet s'inscrit. Ce premier chapitre a pour objectif de poser le cadre du projet. Nous y analyserons la problématique actuelle, identifierons les besoins précis des utilisateurs, et définirons les objectifs fonctionnels à atteindre ainsi que les limites à respecter.

1.1 Description du problème réel :

Dans le cadre d'une gestion bancaire traditionnelle ou semi-informatisée (utilisant de simples registres papier ou des tableurs non sécurisés), l'organisation fait face à des contraintes majeures qui ralentissent son activité et menacent la fiabilité de ses services. Ces défis se déclinent en plusieurs axes critiques :

1. La gestion manuelle et la centralisation des clients Le traitement des dossiers clients sur support physique ou via des fichiers dispersés engendre une lourdeur administrative. La recherche d'informations devient une tâche chronophage, retardant le service au guichet. De plus, ce mode de fonctionnement augmente considérablement le risque d'erreur humaine (doublons, fautes de frappe, perte de fiches) et rend difficile la mise à jour rapide des coordonnées (adresse, téléphone) en temps réel.

2. Le suivi rigoureux des comptes bancaires Sans système automatisé, il est complexe de maintenir une cohérence parfaite entre les comptes et leurs soldes respectifs. La mise à jour différée des soldes après une opération peut entraîner des incohérences, comme autoriser un retrait alors que le solde est insuffisant. Le manque de liaison dynamique entre un client et ses multiples comptes complique également la vue d'ensemble du patrimoine du client.

3. La traçabilité et l'enregistrement des transactions La banque manipule des flux financiers sensibles (dépôts, retraits, virements). L'absence d'un système d'archivage numérique automatique pose un problème de traçabilité. En cas de litige ou d'erreur de caisse, il est extrêmement difficile de retrouver la trace d'une opération spécifique dans des archives manuelles. L'historisation fiable de chaque mouvement financier est pourtant indispensable pour la comptabilité et la transparence.

4. La sécurité et la confidentialité des données Les registres physiques ou les fichiers Excel sont vulnérables. Ils ne permettent souvent pas de définir des niveaux d'accès stricts. Sans mécanisme d'authentification robuste, n'importe quelle personne ayant accès au bureau ou à l'ordinateur peut consulter, modifier, voire détruire des données financières sensibles, compromettant ainsi la confidentialité des clients et l'intégrité de l'institution.

5. La complexité du reporting et de la consultation La génération de rapports (relevés de compte, bilan journalier, liste des clients débiteurs) est une opération fastidieuse lorsqu'elle est faite manuellement. Elle nécessite de compiler des données éparées, ce qui est source d'erreurs de calcul. Les clients et les gestionnaires manquent donc de visibilité immédiate sur l'historique des opérations, ce qui freine la prise de décision et nuit à la qualité de la relation client.

1.2 Les besoins:

L'analyse des problèmes actuels nous a permis de dégager deux catégories de besoins indispensables à la réussite du projet : les besoins fonctionnels (ce que le système doit faire) et les besoins non-fonctionnels (les critères de qualité technique).

1.2.1 Besoins Fonctionnels:

tableau 1: Besoins Fonctionnels

Besoin	Description Détaillée
Gestion des Clients (CRUD)	Le système doit permettre le cycle de vie complet d'un client : ajout d'un nouveau profil, consultation des informations existantes, mise à jour des coordonnées (ex: changement d'adresse) et suppression des clients inactifs.
Gestion des Comptes	Capacité à ouvrir de nouveaux comptes bancaires et à les lier de manière univoque à un client existant. Le système doit garantir qu'un compte appartient toujours à un propriétaire identifié.
Opérations Bancaires	Exécution des transactions financières courantes (retraits, dépôts, virements) avec une mise à jour immédiate et atomique des soldes. Le système doit inclure des règles de gestion (ex: interdire un retrait si le solde est insuffisant).
Authentification Sécurisée	Mise en place d'un mécanisme de contrôle d'accès. Chaque client ou administrateur doit s'identifier via un code (PIN ou mot de passe) avant de pouvoir visualiser ou modifier des données.
Historique & Traçabilité	Enregistrement systématique de toutes les opérations effectuées (date, type, montant). L'utilisateur doit pouvoir consulter un journal (logs) pour suivre l'évolution de son compte.
Interface de Navigation	Le logiciel doit proposer un menu principal clair permettant d'accéder aux différentes fonctionnalités sans ambiguïté, avec une gestion des retours en arrière pour ne pas bloquer l'utilisateur.

1.2.2 Besoins Non-Fonctionnels:

tableau 2 :Besoins Non-Fonctionne

Critère	Description Détaillée
Performance & Réactivité	Le système doit offrir un temps de réponse minimal. Le traitement des données (recherche client, calcul de solde) doit être quasi-instantané pour assurer une fluidité d'utilisation.
Sécurité & Intégrité	Les données sensibles doivent être protégées. Le système doit empêcher les accès non autorisés et garantir que les données ne soient pas corrompues en cas de mauvaise manipulation.
Fiabilité & Persistance	Les données ne doivent pas être perdues à la fermeture du programme. Une sauvegarde automatique dans des fichiers (texte/binaire) est requise pour retrouver l'état exact du système au redémarrage.
Ergonomie (UX)	L'interface doit être intuitive, sobre et facile à prendre en main, même pour un utilisateur novice. Les messages d'erreur doivent être explicites (ex: "Solde insuffisant" au lieu de "Erreur 404").
Portabilité	Le code source doit être standardisé pour permettre une compilation et une exécution sans modification majeure sur différents systèmes d'exploitation (Windows, Linux, macOS).

1.3 Les objectifs fonctionnels:

Pour répondre aux besoins identifiés précédemment, le système proposé devra remplir un ensemble de fonctionnalités clés, garantissant une gestion fluide et sécurisée des activités bancaires.

1. Gestion intégrale des profils clients (CRUD) Le logiciel doit permettre une administration complète de la base de données clients. Cela inclut :

- L'ajout (Create) de nouveaux clients avec la saisie de leurs informations personnelles.
- La consultation (Read) rapide des détails d'un client spécifique.
- La modification (Update) des données en cas de changement (adresse, téléphone).
- La suppression (Delete) des dossiers obsolètes, tout en respectant les règles d'intégrité (ne pas supprimer un client ayant des comptes actifs).

2. Administration des comptes bancaires Le système doit être capable de gérer le cycle de vie des comptes associés à chaque client. L'objectif est de permettre l'ouverture de nouveaux comptes, leur consultation (vérification du solde, type de compte), ainsi que leur fermeture. Chaque compte doit être strictement lié à un propriétaire unique pour assurer la cohérence des données.

3. Exécution sécurisée des opérations bancaires C'est le cœur fonctionnel de l'application. Le système doit gérer les transactions quotidiennes telles que les retraits et les virements (inter-comptes ou vers tiers). Ces opérations ne doivent être validées qu'après des contrôles logiques rigoureux :

- Vérification de la suffisance du solde (pour éviter les découverts non autorisés).
- Validation de l'existence du compte destinataire pour les virements.
- Mise à jour immédiate et atomique des soldes après chaque transaction.

4. Authentification et contrôle d'accès Afin de protéger l'accès aux fonctionnalités et aux données sensibles, une couche de sécurité est indispensable. L'application exigera une authentification préalable (par exemple, via un identifiant et un code PIN) avant d'autoriser l'utilisateur à effectuer des opérations, garantissant ainsi que seules les personnes autorisées peuvent manipuler les comptes.

5. Historisation et traçabilité Pour assurer la transparence des activités, le logiciel doit archiver systématiquement toutes les transactions effectuées. Chaque opération (retrait, virement) sera enregistrée dans un historique consultable, permettant à l'utilisateur ou à l'administrateur de retracer les mouvements financiers passés et de générer des relevés de compte.

6. Interface Utilisateur (UI) et Ergonomie L'interaction avec le système se fera via une interface claire et structurée (menu navigable). Un accent particulier sera mis sur la gestion des erreurs et la robustesse : le programme ne doit pas "planter" en cas de saisie incorrecte, mais plutôt guider l'utilisateur avec des messages explicatifs clairs pour corriger son entrée.

7. Persistance des données Contrairement à une exécution en mémoire volatile, le système doit assurer la pérennité des informations. Toutes les données (clients, comptes, transactions) seront sauvegardées sur des fichiers physiques (fichiers textes ou binaires). Au démarrage, le logiciel devra être capable de charger ces fichiers pour restaurer l'état exact de la banque tel qu'il était à la dernière fermeture.

1.4 Les limites du projet:

Bien que l'application réponde aux objectifs fonctionnels fixés, il est important de définir le cadre de son utilisation. Ce projet étant un prototype [académique / de première version], il comporte certaines limitations techniques et fonctionnelles assumées, décrites ci-dessous :

tableau 3: Les limites du projet

Limitation Identifiée	Description et Justification
Architecture Monoposte	Description : L'application est conçue pour fonctionner sur une machine unique, sans communication réseau. Justification : L'absence de serveur central simplifie l'architecture pour se concentrer sur la logique métier locale plutôt que sur les problématiques réseaux (sockets, protocoles).
Persistance par Fichiers	Description : Les données sont stockées dans des fichiers plats (texte ou binaire) et non dans un SGBD (comme MySQL ou Oracle). Justification : Ce choix permet de démontrer la maîtrise de la manipulation de fichiers bas niveau sans dépendre de bibliothèques tierces complexes.
Interface en Ligne de Commande (CLI)	Description : L'interaction utilisateur se fait via la console (texte), sans interface graphique (fenêtres, souris). Justification : L'objectif prioritaire est le développement des algorithmes de gestion et la robustesse du code backend, plutôt que le design visuel.
Sécurité Simplifiée	Description : Les codes PIN et données sensibles ne font pas l'objet d'un chiffrement cryptographique avancé (stockage en clair ou encodage simple). Justification : Dans cette phase de prototypage, la sécurité est fonctionnelle (contrôle d'accès logique) mais ne répond pas aux normes industrielles bancaires (PCI-DSS).
Absence de Transactions ACID	Description : Le système ne gère pas la concurrence (plusieurs utilisateurs modifiant le même compte simultanément). Justification : La gestion des accès concurrents (verrouillage de fichiers) complexifierait le projet au-delà du périmètre imparti.

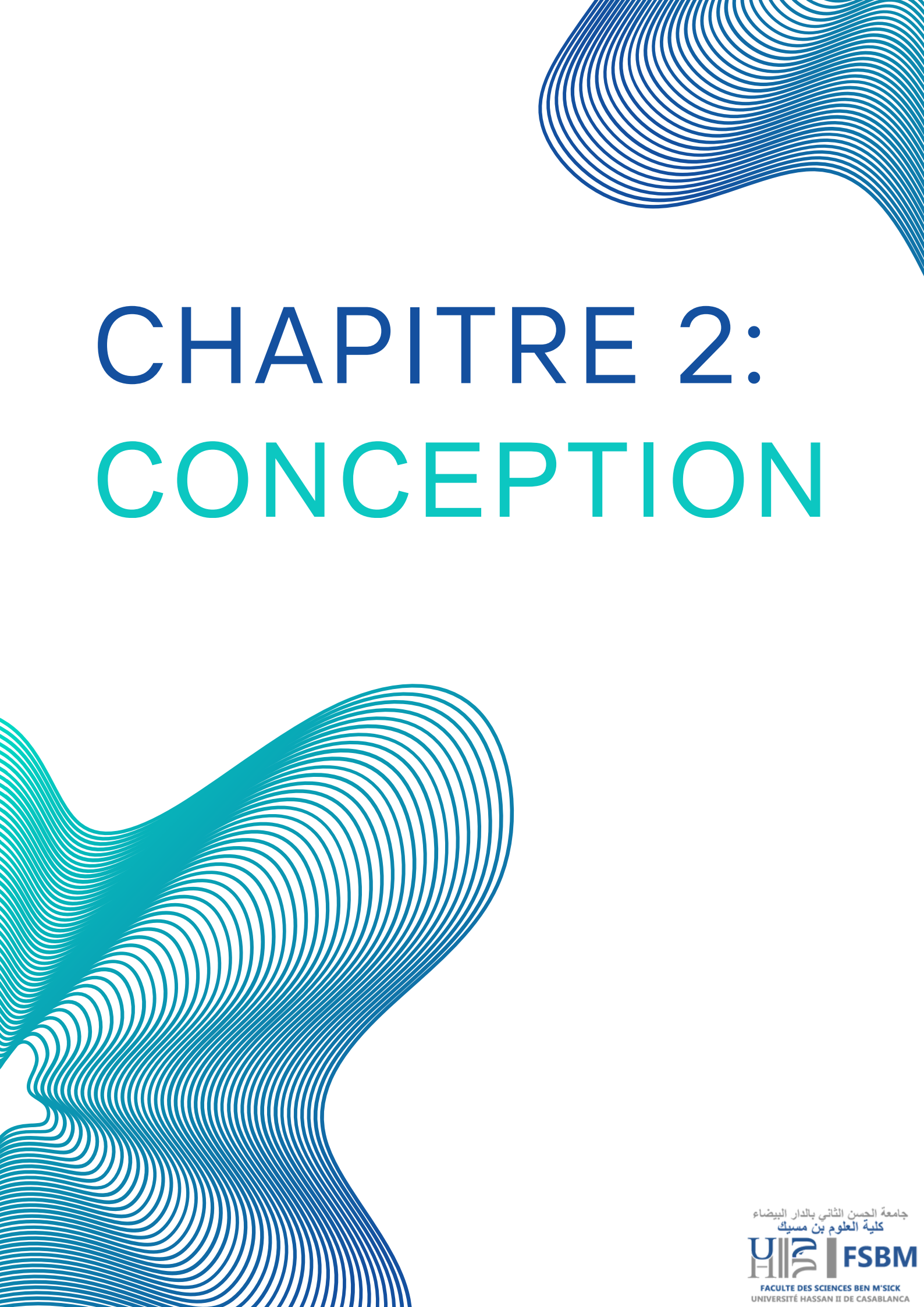
Limitation Identifiée	Description et Justification
Volumétrie Restreinte	Description : Le système est dimensionné pour un nombre fini d'entités (Ex: Max 100 clients/comptes). Justification : Cette limite est liée à l'utilisation de structures de données statiques (tableaux) pour optimiser la gestion de la mémoire dans ce contexte spécifique.
Règles de Gestion Strictes	Description : Plafond imposé de 5000 DH par opération et reporting basique. Justification : Ces règles métier arbitraires permettent de simuler des politiques de risque bancaire et de simplifier les tests de validation.

Conclusion

En conclusion, ce premier chapitre a permis de poser les fondations nécessaires au bon déroulement du projet. Nous avons mis en évidence les limites critiques de la gestion manuelle actuelle et justifié l'impératif de développer une solution informatisée fiable.

En définissant clairement les objectifs fonctionnels (gestion des comptes, transactions, sécurité) ainsi que les contraintes techniques (limites du système), nous avons délimité un périmètre de travail réaliste et maîtrisé. Ce cadrage précis nous permet désormais de passer sereinement à la phase suivante : l'analyse et la conception technique de l'application.

CHAPITRE 2: CONCEPTION



CHAPITRE 2: CONCEPTION

introduction:

Après avoir défini le contexte et les objectifs du projet dans le premier chapitre, nous abordons désormais la phase de conception technique. Cette étape est charnière : elle consiste à traduire les besoins fonctionnels identifiés en une architecture logicielle cohérente, avant même d'écrire la première ligne de code.

Dans ce chapitre, nous détaillerons d'abord l'analyse des besoins pour spécifier le comportement attendu du système. Nous nous appuierons ensuite sur la modélisation UML (diagrammes de cas d'utilisation et de classes) pour visualiser la structure et les interactions du programme. Enfin, nous présenterons les algorithmes principaux qui régissent la logique métier, ainsi que l'organisation modulaire du code

2.1 Analyse détaillée des besoins:

L'analyse des besoins vise à détailler le comportement attendu du système avant toute implémentation. Nous décomposons ici le système en acteurs, flux d'informations et règles de gestion strictes.

2.1.1 Identification des Acteurs:

Le système interagit avec deux types d'utilisateurs ayant des privilèges distincts :

- L'Administrateur (Guichetier) :
 - Il possède les droits d'accès globaux ("Super-User").
 - Il est responsable de la maintenance des données (CRUD Clients et Comptes).
- Le Client :
 - Il possède un accès restreint à ses propres données.
 - Il interagit avec le système pour la consultation et les opérations courantes (simulation d'un GAB ou d'une banque en ligne).

2.1.2 Spécification des Règles de Gestion (RG):

Afin de garantir la fiabilité du système bancaire, l'application doit respecter impérativement les règles suivantes lors de l'exécution du code :

- RG-01 (Unicité) : Deux clients ne peuvent pas posséder le même ID . De même, chaque numéro de compte doit être unique dans tout le système.
- RG-02 (Solde Positif) : Un retrait ou un virement ne peut être validé que si le solde du compte source est supérieur ou égal au montant demandé ($\text{Solde} \geq \text{Montant}$). Aucun découvert n'est autorisé.
- RG-03 (Plafonnement) : Par mesure de sécurité, le montant d'une transaction unique (retrait ou virement) ne peut excéder 5000 DH.
- RG-04 (Format de sécurité) : Le code PIN associé à un client doit être obligatoirement composé de chiffres (entier positif) pour l'authentification.

2.1.3 Analyse des Flux de Données (Entrées/Sorties)

Pour chaque fonctionnalité majeure, nous définissons les données nécessaires et le résultat attendu:

tableau 4:Analyse des Flux de Données (Entrées/Sorties)

Fonctionnalité	Données en Entrée (Input)	Traitement (Logique)	Donnée en Sortie (Output)
Authentification	ID Client + Code PIN	Vérification de la correspondance dans le fichier clients.txt.	Accès au menu ou Message d'erreur.
Création Compte	ID Client + Type de compte	Génération d'un ID unique, initialisation solde à 1000.	Nouveau compte ajouté au fichier.
Virement	ID Source + ID Destinataire + Montant	1. Vérif. existence destinataire. 2. Vérif. solde source. 3. Débit source / Crédit dest.	Confirmation visuelle et mise à jour des soldes.
Recherche Client	CIN ou ID Client	Parcours séquentiel de la liste des clients.	Affichage des détails du client trouvé.

2.1.4 Dictionnaire de Données

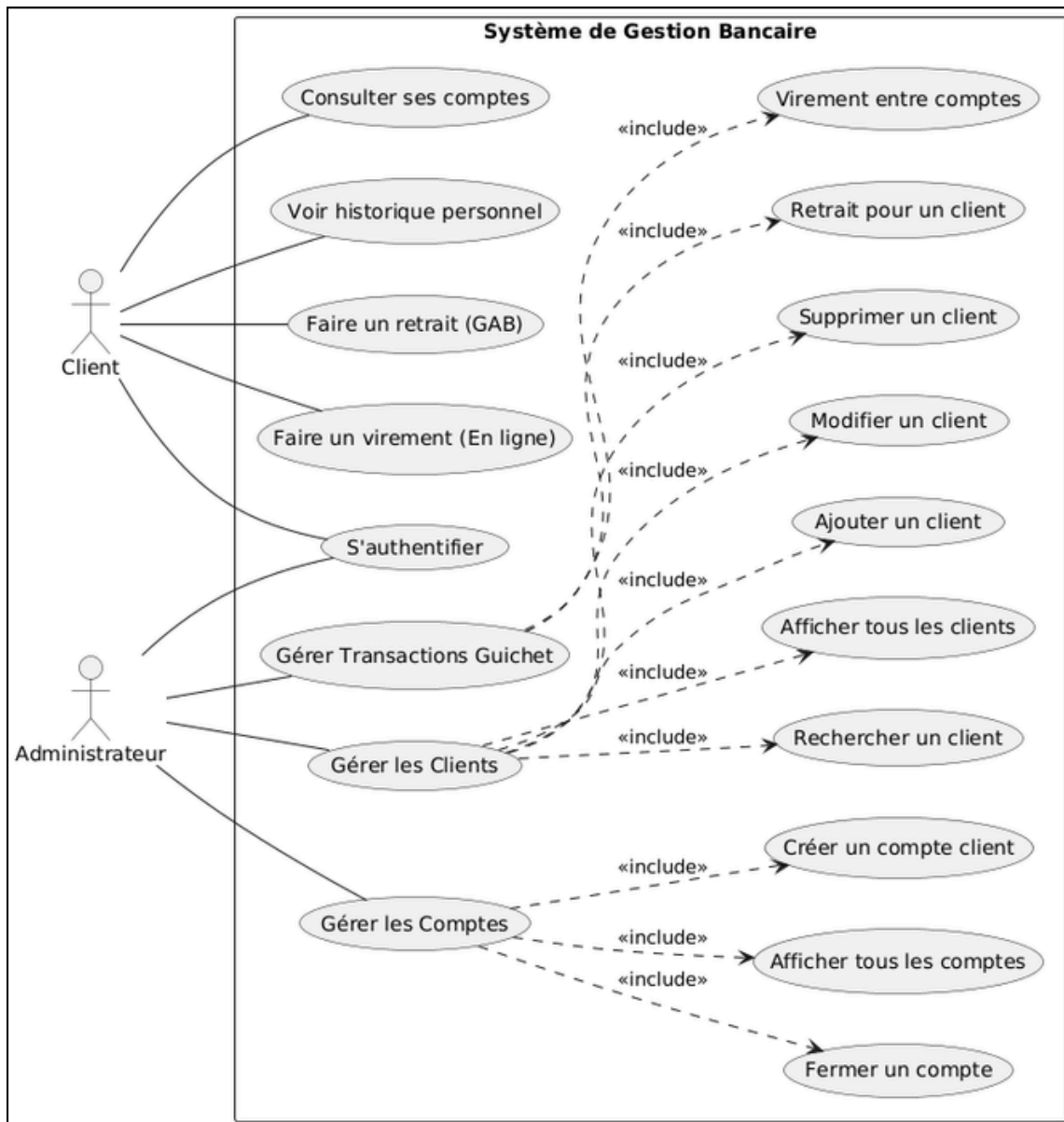
Avant de définir les structures en C, nous listons les informations clés que le système doit mémoriser :

- Entités Clients : Nom, Prénom, ID du client, Téléphone, PIN (Secret).
- Entités Comptes : ID Compte, ID Propriétaire (lien vers Client), Solde (Montant réel), Date de création.
- Entités Transactions : Type (Retrait/Virement), Montant, Date de l'opération

2.2 Les Diagrammes (UML):

2.2.1 diagramme de cas d'utilisation:

figure 1:diagramme de cas d'utilisation

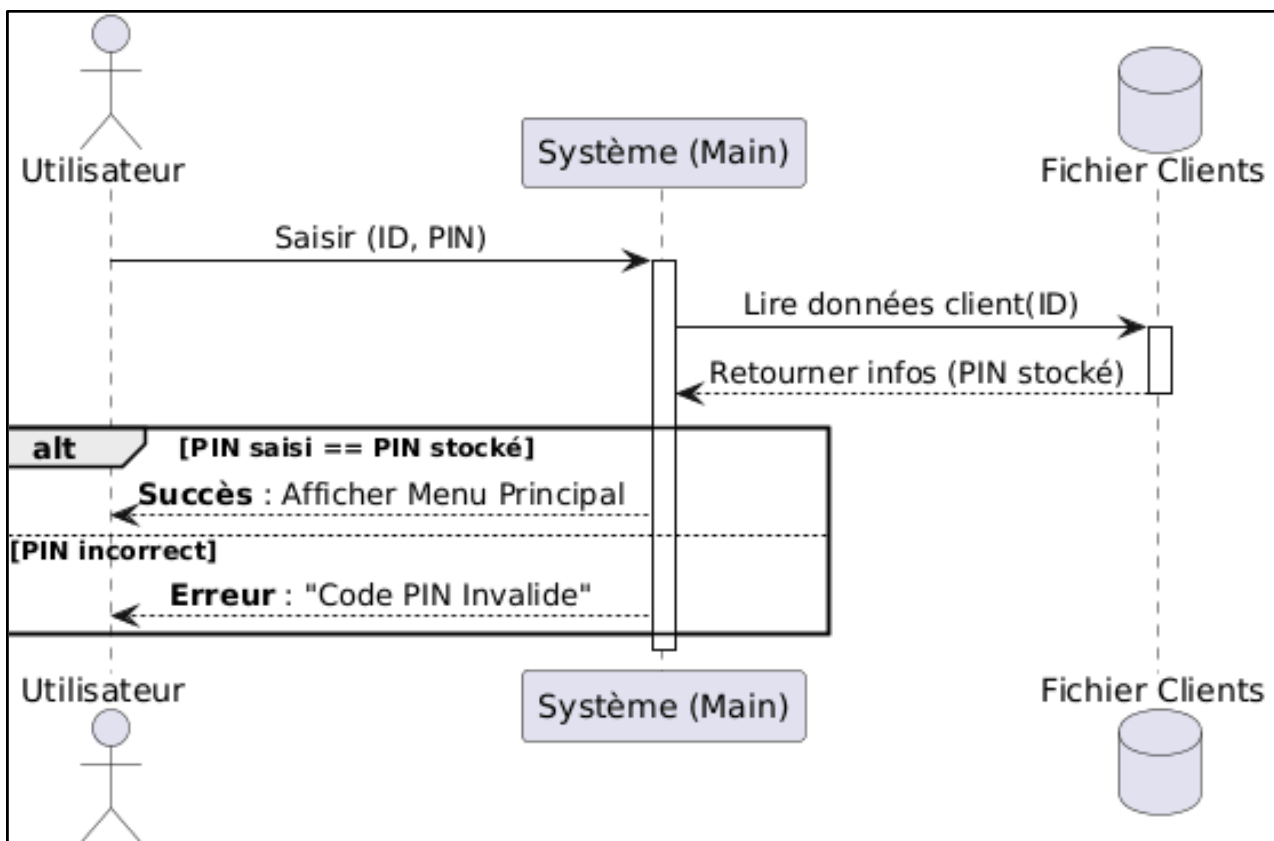


Ce diagramme de classes propose une architecture modulaire adaptée au langage C, séparant clairement les structures de données statiques de la logique fonctionnelle du système. La couche Données définit les trois entités fondamentales : le Client (identifié par ses informations personnelles et son PIN), qui entretient une relation de multiplicité 'un-à-plusieurs' avec le Compte bancaire, lequel encapsule par composition un historique de Transactions.

En parallèle, la couche Logique Métier organise les traitements en trois services distincts : le ServiceAuthentification qui sécurise l'accès, le ServiceAdmin qui centralise les privilèges élevés (gestion complète du cycle de vie des clients et des comptes, ainsi que les opérations de guichet), et le ServiceClient qui offre des fonctionnalités restreintes d'autonomie (consultation de solde, retraits GAB). Les relations de dépendance (flèches en pointillés) illustrent comment ces modules fonctionnels manipulent les structures de données pour lire l'état de la banque ou modifier les soldes en temps réel.

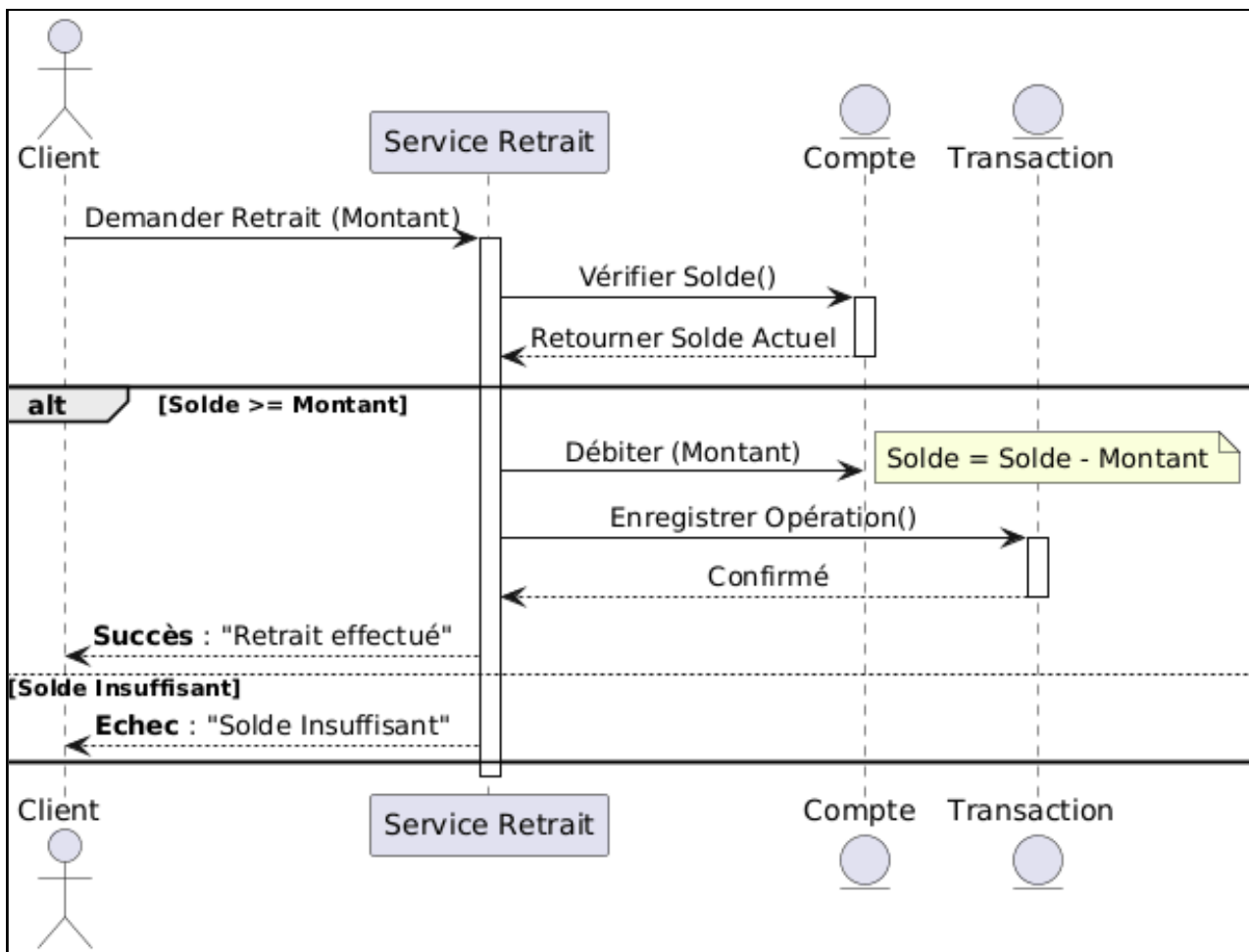
2.2.2 les diagrammes de sequence

figure 2:diagramme de sequence processus de connexion



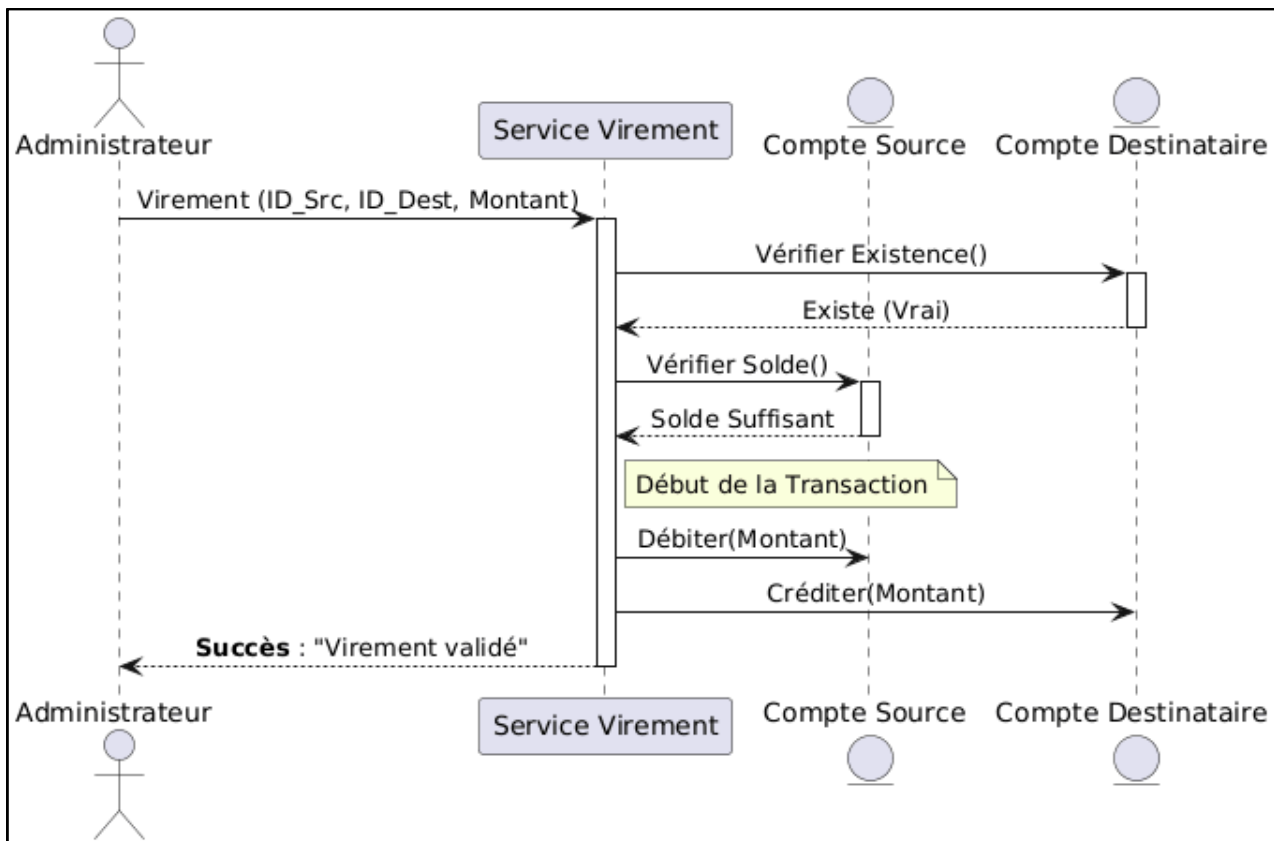
Ce diagramme détaille le processus de connexion. Le système interroge le fichier de données pour récupérer le PIN associé à l'identifiant saisi. Une comparaison est ensuite effectuée pour valider ou refuser l'accès

figure3::diagramme de sequence la logique de sécurité lors d'un retrait



Ce diagramme illustre la logique de sécurité lors d'un retrait. Avant toute modification, le système compare le montant demandé au solde disponible. Si la condition est remplie, le compte est débité et l'opération est historisée

figure4::diagramme de sequence Le scénario de virement



Le scénario de virement met en évidence l'interaction entre deux entités distinctes. Le système doit s'assurer séquentiellement que le compte destinataire existe et que le compte source est provisionné avant d'opérer le transfert de fonds.

2.3 Algorithmes Principaux:

2.3.1 algorithme d authentification:

figure5:algorithme d authentification

```

ALGORITHME: Authentifier_Client
ENTRÉE: ID du client, code PIN
DÉBUT
  1. Lire ID depuis console
  2. SI ID == "cancel" ALORS
    - Retourner à menu
  3. Chercher client avec cet ID
  4. SI client non trouvé ALORS
    - Afficher erreur
    - Attendre et réessayer
  
```

```
5. Demander le code PIN
6. SI PIN correct ALORS
    - Enregistrer logged_in_client_id
    - Afficher bienvenue
7. SINON
    - Afficher erreur authentication
    - Réessayer
FIN
```

L'objectif de cette procédure est de vérifier l'identité de l'utilisateur avant de lui donner accès aux fonctionnalités sensibles (retrait, virement). Elle assure que seule la personne connaissant le code PIN associé à un ID spécifique peut se connecter.

- Saisie de l'Identifiant : Le système demande d'abord à l'utilisateur de s'identifier.
 - Fonctionnalité d'annulation : L'étape 2 prévoit une porte de sortie. Si l'utilisateur tape "cancel" (ou 0), l'algorithme s'arrête et retourne au menu principal. C'est essentiel pour ne pas "piéger" un utilisateur qui aurait cliqué sur "Connexion" par erreur.
- Vérification de l'existence (Recherche) : À l'étape 3, le programme parcourt la base de données (tableau ou fichier) pour voir si cet ID existe.
 - Cas d'erreur : Si l'ID est introuvable, inutile de demander le PIN. Le système signale l'erreur immédiatement.
- Vérification du Secret (Authentication) : Une fois le client trouvé, le système demande le code PIN (étape 5). Il compare alors :
 - Le PIN saisi au clavier.
 - Le PIN stocké dans la structure Client trouvée à l'étape précédente.
- Gestion de la Session (Succès) : Si les codes correspondent, la variable critique `logged_in_client_id` est mise à jour.
 - Pourquoi c'est important ? Cette variable servira de "mémoire" pour la suite du programme. Quand le client demandera "Voir mon solde", le programme utilisera `logged_in_client_id` pour savoir quel compte afficher.
- Gestion des Échecs : Si le PIN est incorrect, l'accès est refusé. L'algorithme mentionne "Réessayer", ce qui implique généralement une boucle (par exemple, 3 tentatives maximum avant blocage) pour éviter les attaques par force brute.

2.3.2 Algorithme de Retrait:

figure6:Algorithme de Retrait

```
ALGORITHME: Effectuer_Retrait
ENTRÉE: ID client, montant
DÉBUT
  1. Chercher client
  2. SI client non trouvé ALORS
    - Afficher erreur et réessayer
  3. Afficher les comptes du client
  4. Client sélectionne compte avec flèches
  5. Demander le montant
  6. Valider montant:
    - > 0
    - <= plafond (5000 DH)
    - <= solde disponible
  7. SI montant invalide ALORS
    - Afficher erreur, réessayer
  8. Demander confirmation
  9. SI confirmer ALORS
    - Déduire montant du solde
    - Sauvegarder transaction
    - Afficher succès
  10. SINON
    - Annuler l'opération
FIN
```

Cette procédure permet de débiter un montant d'un compte spécifique. Elle assure la solvabilité du client (a-t-il assez d'argent ?) et le respect des règles de la banque (plafond) avant toute modification.

Phase A : Sélection (Étapes 1 à 4) L'algorithme commence par identifier le client concerné. Une fois trouvé, le système affiche la liste de ses comptes bancaires.

- Ergonomie : L'étape 4 mentionne une "sélection avec flèches". Plutôt que de demander à l'utilisateur de taper un ID de compte (source d'erreurs), le système propose une navigation visuelle pour choisir le compte à débiter.

Phase B : Validation des Règles Métier (Étapes 5 à 7) C'est le cœur sécuritaire de l'algorithme. Une fois le montant saisi, il passe par trois filtres obligatoires :

- Cohérence mathématique : Le montant doit être strictement positif (> 0).
- Gestion du risque (Plafond) : Le montant ne doit pas dépasser la limite fixée par la banque (5000 DH dans ce projet).
- Vérification de solvabilité : Le solde actuel du compte doit être suffisant pour couvrir le retrait ($\text{Solde} \geq \text{Montant}$).

Si une seule de ces conditions n'est pas remplie, l'opération est bloquée.

Phase C : Exécution et Traçabilité (Étapes 8 à 10) Avant de valider, le système demande une confirmation explicite (Oui/Non) pour éviter les erreurs de frappe. Si l'utilisateur confirme :

- Mise à jour : Le solde est diminué du montant.
- Traçabilité : Une ligne est écrite dans le fichier des transactions (Historique) pour garder une preuve de l'opération.

Feedback : Un message de succès informe l'utilisateur

2.3.3 Algorithme de Recherche Client:

figure7:Algorithme de Recherche Client

```
ALGORITHME: Rechercher_Client
ENTRÉE: Critère de recherche (ID ou nom)
DÉBUT
  1. Proposer choix: Recherche par ID ou Nom
  2. SI par ID ALORS
    - Lire ID
    - Chercher correspondance exacte
  3. SI par Nom ALORS
    - Lire nom/prénom
    - Chercher correspondance partielle (case-insensitive)
    - Afficher tous les résultats
  4. SI aucun résultat ALORS
    - Afficher "non trouvé"
    - Proposer réessayer
  5. SINON
    - Afficher résultats trouvés
    - Si un seul: afficher détails
    - Si plusieurs: afficher liste
FIN
```

Cette procédure vise à extraire un ou plusieurs profils clients de la base de données en fonction d'un critère saisi par l'utilisateur. Elle est conçue pour être polyvalente, offrant à la fois une recherche précise (technique) et une recherche intuitive (humaine).

2. Déroulement Logique par étapes

Phase A : Choix de la Méthode (Étape 1) Le système ne se limite pas à une seule méthode. Il propose une bifurcation dès le début :

- Recherche par ID : Pour une identification rapide et sans équivoque (souvent utilisée quand le client donne son numéro de dossier).
- Recherche par Nom : Pour une recherche plus naturelle (utilisée quand le client déclare son identité orale).

Phase B : Traitement des Critères (Étapes 2 et 3) C'est ici que la logique diffère selon le choix :

- Par ID (Recherche Exacte) : L'algorithme cherche une égalité parfaite (ID_saisi == ID_stocké). Comme l'ID est unique, la recherche s'arrête dès que le client est trouvé.
- Par Nom (Recherche Souple) : L'algorithme est plus intelligent.
 - Insensible à la casse : Il ne fait pas de distinction entre majuscules et minuscules (trouve "ALAMI" même si on tape "alami").
 - Correspondance partielle : Il vérifie si la saisie est une partie du nom (ex: taper "Ben" affichera "Benjelloun" et "Benali").

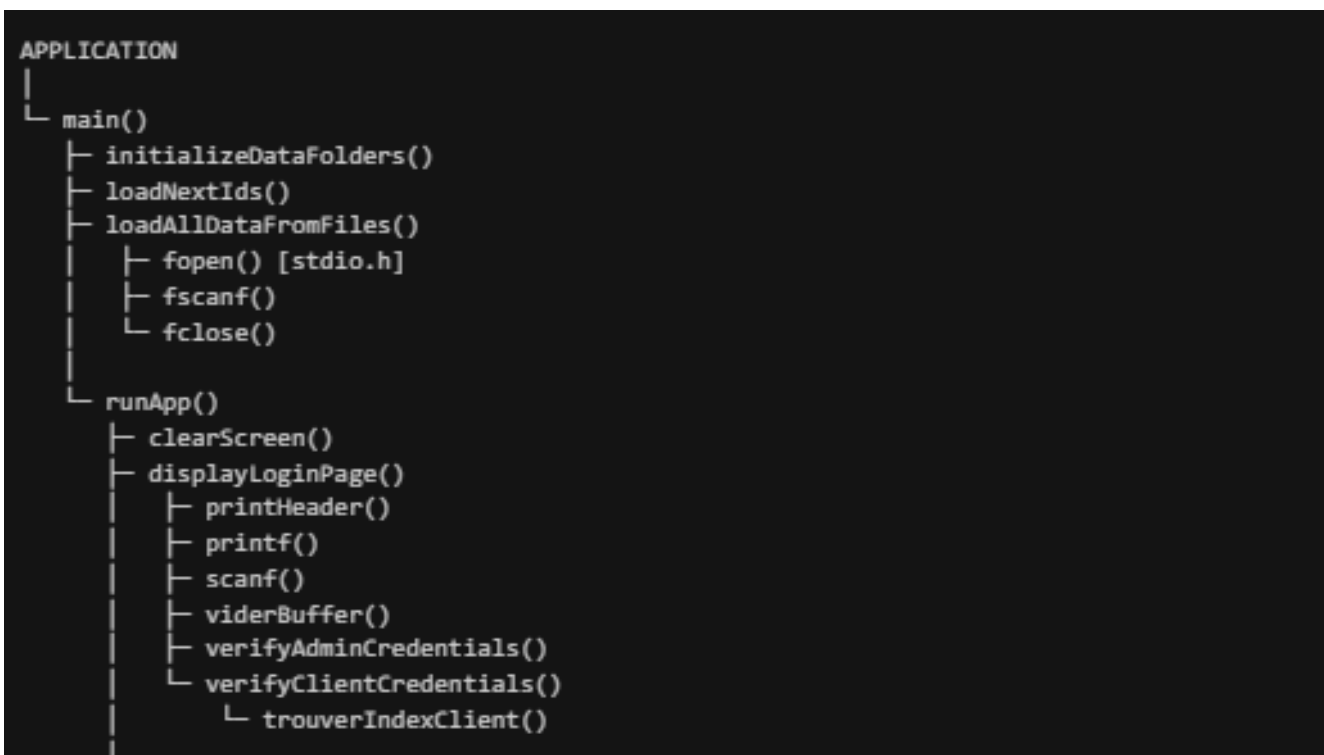
Phase C : Affichage Intelligent des Résultats (Étapes 4 et 5) L'algorithme gère la sortie de manière dynamique :

- Aucun résultat : Un message clair informe l'utilisateur, évitant la confusion.
- Un seul résultat : (Cas typique de l'ID) Le système affiche directement la fiche détaillée du client.
- Plusieurs résultats : (Cas typique du Nom) Si on cherche "Mohamed", il y en a probablement plusieurs. Le système affiche alors une liste intermédiaire pour permettre à l'administrateur de choisir le bon profil.

2.4 Structure du programme:

2.4.1 Les Fonctions:

figure8:Les Fonctions:



```

└─ adminDashboard()
    └─ displayMainMenu()
        └─ printHeader()
        └─ printf()
        └─ demanderConfirmationAvecFleches()
        └─ waitForEnter()

    └─ displayClientsMenu() → [GESTION CLIENTS]
        └─ ajouterClient()
        └─ modifierClient()
        └─ supprimerClient()
        └─ rechercherClient()
        └─ afficherTousClients()

    └─ displayAccountsMenu() → [GESTION COMPTES]
        └─ ajouterCompte()
        └─ consulterComptes()
        └─ fermerCompte()

    └─ displayOperationsMenu() → [OPÉRATIONS BANCAIRES]
        └─ effectuerRetrait()
        └─ effectuerVirement()

    └─ displayAide()
    └─ waitForEnter()

└─ clientDashboard()
    └─ displayClientMenu()
    └─ displayClientAccountsMenu()
    └─ displayClientWithdrawalMenu() → clientRetrait()
    └─ displayClientTransferMenu() → clientVirement()
    └─ displayClientHistoryMenu() → afficherHistoriquePourClient()
    └─ resetClientPIN()
    └─ waitForEnter()
    
```

L'architecture fonctionnelle de l'application s'articule autour d'une hiérarchie d'appels rigoureuse, pilotée par la fonction principale `main()` qui assure d'abord l'initialisation des dossiers et le chargement des données en mémoire via `loadAllDataFromFiles()`. Le flux d'exécution bascule ensuite vers la boucle `runApp()`, véritable chef d'orchestre de l'interface, qui gère l'authentification sécurisée des utilisateurs. Selon le rôle validé, le programme redirige le flux soit vers le `adminDashboard()` pour l'accès aux modules de gestion (clients, comptes, opérations), soit vers le `clientDashboard()` pour les services d'autonomie bancaire, garantissant ainsi une séparation stricte des privilèges et une navigation fluide entre les différents modules fonctionnels.

2.4.2 structures et Leurs Rôles:

2.4.2.1 structure Client:

figure9:structure Client

```
typedef struct {
    int id_client;           // Identifiant unique
    char nom[50];           // Nom de famille
    char prenom[50];        // Prénom
    char profession[50];    // Profession/Métier
    char num_tel[15];       // Numéro de téléphone
    char code_pin[20];      // Code PIN authentication
} Client;
```

tableau 5:structure Client

Champ	Type	Taille	Rôle	Description
id_client	int	4 octets	Clé Primaire	Identifiant unique auto-généré pour chaque client (1, 2, 3...). Commence à 1 et s'incrémente.
nom	char[50]	50 octets	Identification	Nom de famille du client. Max 49 caractères + \0. Contient seulement alphabets et espaces.
prenom	char[50]	50 octets	Identification	Prénom du client. Max 49 caractères + \0. Contient seulement alphabets et espaces.

Champ	Type	Taille	Rôle	Description
profession	char[50]	50 octets	Information	Profession ou métier du client. Max 49 caractères + \0. Texte libre.
num_tel	char[15]	15 octets	Contact	Numéro de téléphone unique. Exactement 10 chiffres. Max 14 caractères + \0. Pas de doublon autorisé.
code_pin	char[20]	20 octets	Sécurité	Code PIN d'authentification. 4 à 10 chiffres. Max 19 caractères + \0. Utilisé pour connexion.

2.4.2.2 structure Compte:

figure10:structure Compte

```
typedef struct {
    int id_compte;           // Identifiant unique
    int id_client;          // Référence au client
    float solde;             // Solde actuel
    char date_ouverture[11]; // Date d'ouverture
} Compte;
```

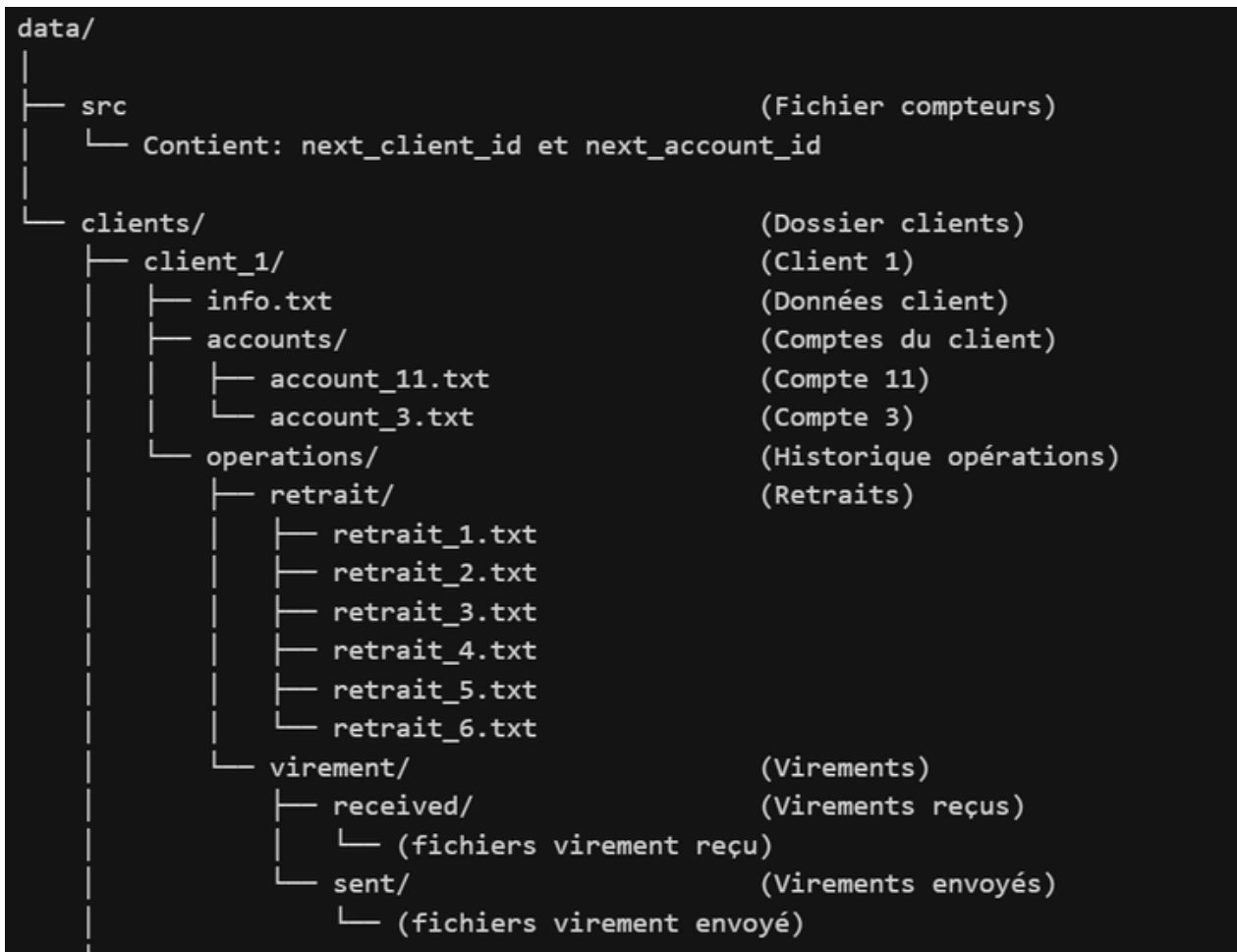
tableau 6:structure Compte

Champ	Type	Taille	Rôle	Description
id_compte	int	4 octets	Clé Primaire	Identifiant unique auto-généré pour chaque compte (1, 2, 3...). Commence à 1 et s'incrmente.
id_client	int	4 octets	Clé Étrangère	Référence au client propriétaire (lien avec Client.id_client). Utilisé pour retrouver le propriétaire.
solde	float	4 octets	État Financier	Solde actuel en DH. Montant décimal. Minimum 1000 DH à création. Peut être 0 avant fermeture.
date_ouverture	char[11]	11 octets	Audit	Date d'ouverture du compte. Format: JJ/MM/AAAA (10 chars + \0). Enregistrée à la création.

2.4.3 Les fichiers:

2.4.3.1 Stockage des données:

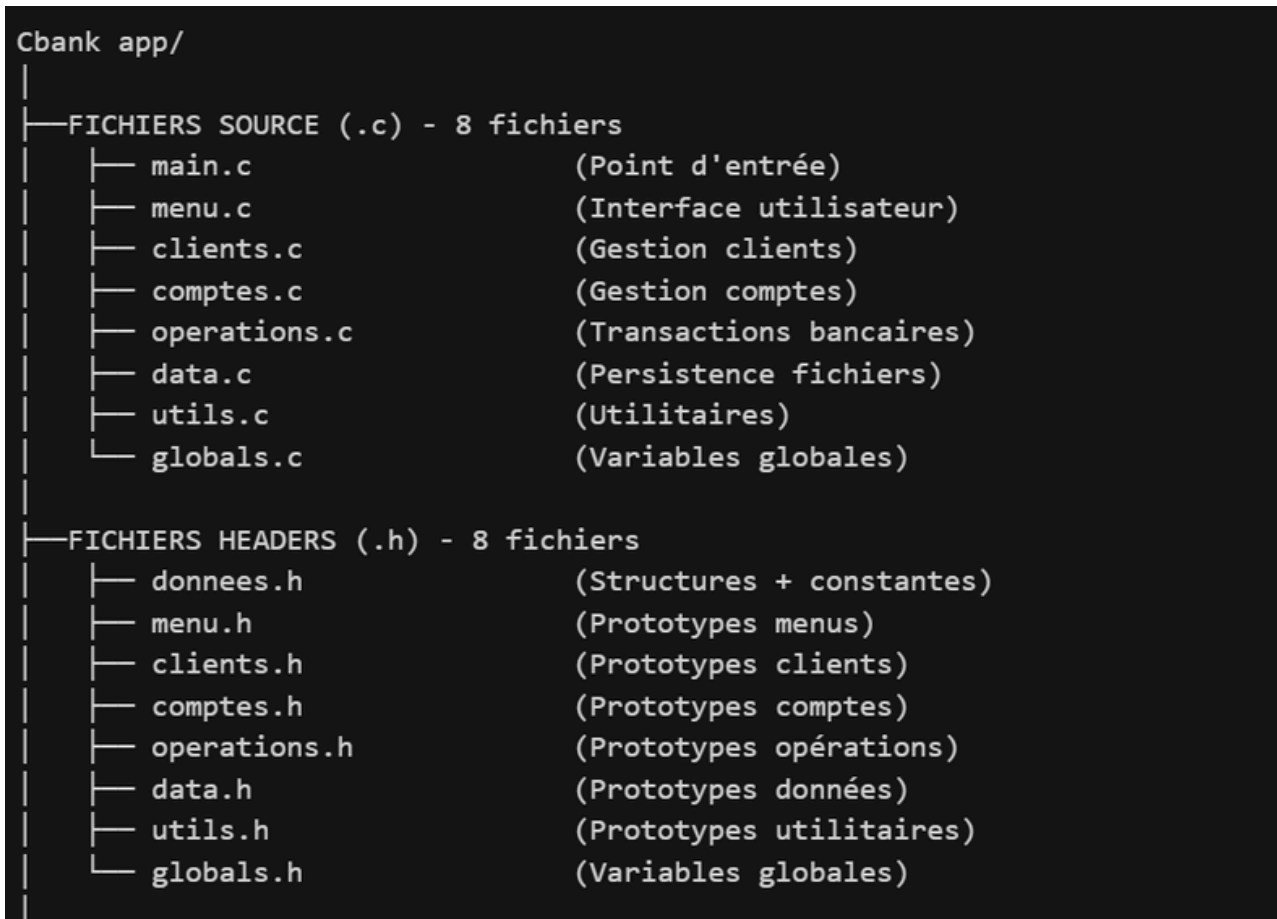
figure11:Stockage des données



La persistance des données est assurée par une architecture de fichiers hiérarchique et modulaire, centralisée dans un répertoire racine data/. Ce dernier abrite un fichier de contrôle src pour la gestion des identifiants uniques (auto-incrémentation) et un dossier clients/ où chaque utilisateur dispose d'un espace de stockage isolé. La structure interne de chaque dossier client organise rigoureusement les informations : un fichier info.txt pour les données personnelles, un sous-répertoire accounts/ contenant chaque compte bancaire sous forme de fichier distinct, et un dossier operations/ classant méticuleusement les historiques de retraits et de virements (envoyés et reçus), garantissant ainsi une traçabilité optimale et une intégrité des données sans recours à une base de données relationnelle.

2.4.3.2 Structure des fichiers de projet:

figure12: Structure des fichiers de proje



conclusion:

En conclusion, ce chapitre nous a permis de traduire les besoins fonctionnels identifiés en une solution technique concrète et structurée. Grâce à la modélisation UML (diagrammes de cas d'utilisation et de classes), nous avons défini clairement les interactions entre les acteurs et l'architecture des données. De plus, l'élaboration des algorithmes critiques et le découpage modulaire du programme garantissent une logique métier robuste avant même le début du développement.

Forts de cette conception détaillée, qui sert désormais de feuille de route technique, nous pouvons aborder sereinement la phase de Réalisation. Le chapitre suivant sera consacré à l'implémentation du code, à la traduction de ces structures en langage C, et à la présentation des interfaces finales de l'application

CHAPITRE 3:

RÉALISATION



CHAPITRE 3: RÉALISATION

introduction:

Après avoir défini l'architecture et le fonctionnement théorique de notre application dans le chapitre précédent, nous passons maintenant à l'étape la plus concrète : la réalisation technique. Ce chapitre marque le passage du papier à l'écran, où nos diagrammes UML et nos algorithmes prennent vie sous forme de code.

Le développement en langage C a été une étape cruciale qui nous a demandé bien plus que de la simple écriture de code. Il a fallu structurer le projet intelligemment, gérer la mémoire manuellement et assurer la persistance des données dans des fichiers pour éviter de tout perdre à chaque redémarrage. Nous avons également dû relever le défi de créer une interface console agréable et navigable, ce qui n'est pas natif en C. C'est ici que la logique métier imaginée plus tôt se confronte aux contraintes réelles de la machine.

Nous commencerons par détailler notre implémentation en C (3.1), en montrant comment nous avons organisé nos fichiers et nos fonctions. Ensuite, nous présenterons rapidement l'environnement de travail (3.2) et les outils qui nous ont permis de compiler le projet. Enfin, nous terminerons par la partie résultats (3.3), où des captures d'écran viendront prouver le bon fonctionnement de l'application à travers différents tests.

3.1. Implémentation en C:

3.1.1 Les Fonctions ,explication et description:

tableau 7:Les Fonctions ,explication et description

Fichier	Fonction	Explication
main.c	main()	Initialise les dossiers de données, charge les compteurs ID et les données existantes depuis les fichiers. Lance la boucle principale runApp() qui gère l'interaction avec l'utilisateur.
menu.c	runApp()	Boucle infinie qui gère l'affichage de la page de connexion et la redirection vers les menus appropriés. Continue jusqu'à la déconnexion de l'utilisateur.

Fichier	Fonction	Explication
menu.c	displayLoginPage()	Affiche l'interface de connexion et demande l'ID et le mot de passe/PIN. Vérifie les credentials (admin ou client) et retourne 1 si succès, 0 sinon
menu.c	adminDashboard()	Affiche le menu principal pour l'administrateur avec les options de gestion disponibles. Gère la navigation et le routage vers les sous-menus (gestion clients, comptes, opérations).
menu.c	clientDashboard()	Affiche le menu personnalisé pour le client connecté avec options limitées. Permet consultation de comptes, retraits, virements, historique et changement de PIN.
menu.c	displayMainMenu()	Affiche le menu principal admin avec les 5 options principales (Clients, Comptes, Opérations, Aide, Déconnexion). Gère la sélection avec les flèches directionnelles et valide avec ENTREE.
menu.c	displayClientMenu()	Affiche le menu client avec les options disponibles pour un client non-administrateur. Retourne le code de l'option sélectionnée pour routage vers la fonction appropriée.

Fichier	Fonction	Explication
menu.c	displayClientsMenu()	Affiche le sous-menu de gestion des clients avec 5 options (Ajouter, Modifier, Supprimer, Rechercher, Afficher tous). Route vers la fonction correspondante après sélection.
menu.c	displayAccountsMenu()	Affiche le sous-menu de gestion des comptes avec 3 options (Créer, Consulter, Fermer). Route vers la fonction correspondante pour la gestion des comptes bancaires.
menu.c	displayOperationsMenu() ()	Affiche le sous-menu des opérations bancaires avec 3 options (Retrait, Virement, Historique). Route vers la fonction correspondante pour effectuer les transactions.
menu.c	displayAide()	Affiche un guide d'utilisation de l'application avec instructions et fonctionnalités disponibles. Permet à l'utilisateur de comprendre comment utiliser le système.

menu.c	displayClientAccountsMenu()	Liste tous les comptes du client connecté avec leur solde et date d'ouverture. Permet de consulter les détails d'un compte en particulier
menu.c	displayClientWithdrawalMenu()	Affiche les comptes disponibles du client pour la sélection du compte source du retrait. Permet la sélection avec flèches et valide le montant avant effectuation.
menu.c	displayClientTransferMenu()	Affiche les comptes source disponibles et demande la saisie du compte destination. Gère la sélection du compte et la validation avant effectuation du virement.
menu.c	displayClientHistoryMenu()	Affiche l'historique complet de toutes les transactions du client connecté (retraits et virements). Trie les opérations par date décroissante (plus récent en premier).
menu.c	resetClientPIN()	Demande au client son ancien PIN pour vérification puis le nouveau PIN souhaité. Valide et sauvegarde le nouveau code PIN en mémoire et dans les fichiers.
menu.c	selectClientAccount()	Helper pour sélectionner un compte parmi les comptes du client avec navigation flèches. Retourne l'index du compte sélectionné ou -1 si annulation.
menu.c	clearScreen()	Exécute la commande système cls pour effacer l'écran console sur Windows. Rafraîchit l'affichage et améliore l'expérience utilisateur.

menu.c	printHeader()	Affiche un bandeau standardisé avec le titre passé en paramètre et 91 caractères séparateurs. Utilise la couleur cyan (ANSI) pour une meilleure présentation.
menu.c	printFooter()	Affiche le message standard "Appuyez sur ENTREE pour continuer..." avec séparateurs. Signale à l'utilisateur la fin d'une page et l'attente de confirmation.
menu.c	printBox()	Affiche un message texte formaté avec le préfixe >> et couleur info. Utilisé pour afficher des notifications, avertissements ou messages informatifs.
menu.c	waitForEnter()	Affiche le footer et attend que l'utilisateur appuie sur ENTREE. Efface l'écran après confirmation pour la prochaine page.
menu.c	demanderConfirmationAvecFleches()	Affiche 2 options (Oui/Non) et gère la sélection avec les flèches haut/bas avec wrapping automatique. Retourne 'O' pour Oui ou 'N' pour Non après validation avec ENTREE.

clients.c	ajouterClient()	Demande les informations du nouveau client (nom, prénom, profession, téléphone, PIN). Valide les données et sauvegarde le client en mémoire et fichier avec ID auto-généré.
clients.c	modifierClient()	Demande l'ID du client à modifier et affiche les informations actuelles. Permet de modifier un ou plusieurs champs (nom, prénom, profession, téléphone, PIN).
clients.c	supprimerClient()	Demande l'ID du client à supprimer et contrôle qu'il n'a pas de comptes actifs. Demande confirmation et supprime le client de la mémoire et des fichiers.
clients.c	rechercherClient()	Permet la recherche par ID (exacte) ou par nom (partielle et case-insensitive). Affiche les résultats trouvés et les détails du client si recherche unique.

clients.c	afficherTousClients()	Affiche tous les clients en mémoire dans un tableau formaté avec colonnes normalisées. Trie les clients par ID croissant pour meilleure lisibilité.
comptes.c	ajouterCompte()	Demande l'ID du client propriétaire et le solde initial (minimum 1000 DH). Génère un ID unique et sauvegarde le compte avec la date d'ouverture actuelle.
comptes.c	consulterComptes()	Affiche tous les comptes d'un client spécifique en tableau formaté avec solde et date. Permet consulter l'historique transactionnel d'un compte choisi.
comptes.c	fermerCompte()	Demande l'ID du compte à clôturer et vérifie que le solde est zéro. Après confirmation, supprime le compte de la mémoire et des fichiers.
comptes.c	afficherHistoriquePourCompte()	Charge les transactions (retraits et virements) associées au compte spécifié. Affiche en tableau trié par date décroissante avec type d'opération et montant.
operations.c	effectuerRetrait()	Administrateur sélectionne un client et un compte avec navigation flèches pour effectuer un retrait. Valide le montant (>0 , ≤ 5000 DH, $\leq$ solde) et enregistre la transaction.

operations.c	effectuerVirement()	Administrateur transfère de l'argent entre deux comptes quelconques avec sélection flèches. Valide que les comptes sont différents, débit/crédite correctement et enregistre bilatéralement.
operations.c	clientRetrait()	Client effectue un retrait sur son propre compte avec sélection si plusieurs comptes. Valide le montant et enregistre la transaction dans l'historique personnel.
operations.c	clientVirement()	Client transfère de l'argent depuis son compte vers un autre compte quelconque. Valide les conditions (compte différent, montant valide) et enregistre bilatéralement.
operations.c	afficherHistoriquePourClient()	Charge et fusionne tous les retraits et virements du client connecté. Affiche en tableau fusionné trié par date avec type d'opération et montant.
data.c	initializeDataFolders()	Crée la structure de dossiers data/, data/clients/, data/retraits/, data/virements/ si elle n'existe pas. Utilise l'API Windows _mkdir() pour créer les répertoires.
data.c	loadAllDataFromFiles()	Lit le fichier data/clients/src et charge tous les clients en mémoire. Lit le fichier data/comptes.txt et charge tous les comptes en mémoire avec compteurs.

data.c	saveClient()	Réécrit le fichier data/clients/src avec tous les clients actuels en mémoire. Format CSV: ID,Nom,Prénom,Profession,Tél,PIN. Remplace fichier existant
data.c	saveAccount()	Réécrit le fichier data/comptes.txt avec tous les comptes actuels en mémoire. Format CSV: IDCompte,IDClient,Solde,DateOuverture. Remplace fichier existant.
data.c	saveWithdrawal()	Ajoute une ligne dans data/retraits/client_[ID].txt (append-only). Enregistrement: IDCompte,Montant,DateHeure. Un fichier par client pour traçabilité.
data.c	saveTransferSent()	Ajoute une ligne dans data/virements/client_[ID].txt marquée comme ENVOYÉ (append-only). Enregistrement: IDCompteFrom,IDCompteTo,Montant,DateHeure,ENVOYÉ. Trace l'envoi.
data.c	saveTransferReceived()	Ajoute une ligne dans data/virements/client_[ID].txt marquée comme REÇU (append-only). Enregistrement: IDCompteFrom,IDCompteTo,Montant,DateHeure,REÇU. Trace la réception.
data.c	deleteClientFolder()	Supprime les fichiers d'historique du client: data/retraits/client_[ID].txt et data/virements/client_[ID].txt. Nettoyage lors de la suppression d'un client du système.

data.c	deleteAccountFile()	Réécrit le fichier data/comptes.txt sans le compte supprimé. Supprime logiquement le compte de la persistance de données.
utils.c	viderBuffer()	Nettoie les caractères résiduels dans le buffer d'entrée standard (stdin). Prévient les problèmes de \n affectant la prochaine saisie après scanf()
utils.c	trouverIndexClient()	Parcourt l'array clients[] et cherche le client avec l'ID spécifié. Retourne l'index dans l'array ou -1 si non trouvé. Complexité O(n).
utils.c	trouverIndexCompte()	Parcourt l'array comptes[] et cherche le compte avec l'ID spécifié. Retourne l'index dans l'array ou -1 si non trouvé. Complexité O(n).
utils.c	isCancelInput()	Vérifie si la chaîne entrée par l'utilisateur est "cancel" (case-insensitive). Utilisé pour permettre l'annulation d'une opération en cours.
utils.c	isValidName()	Valide qu'un nom/prénom contient uniquement des alphabets (a-z, A-Z) et espaces. Refuse les chiffres, caractères spéciaux et vérifie la longueur.

utils.c	isValidPhoneNumber()	Valide qu'un numéro de téléphone contient exactement 10 chiffres et pas de caractères spéciaux. Vérifie également l'unicité: le numéro ne doit pas déjà exister dans la base
utils.c	isValidPIN()	Valide qu'un PIN contient uniquement des chiffres et a une longueur entre 4 et 10 caractères. Refuse les espaces et caractères non-numériques.
utils.c	verifyAdminCredentials()	Compare l'ID et le password fournis avec les credentials administrateur hardcodés (admin/admin123). Retourne 1 si credentials corrects, 0 sinon.
utils.c	verifyClientCredentials()	Trouve le client par ID et compare le PIN fourni avec le PIN stocké. Retourne 1 si PIN correct, 0 sinon. Authentification client.
utils.c	displayCancelNotification()	Affiche le message "Opération annulée" avec couleur warning et séparateurs standardisés. Pause 1 seconde et efface l'écran pour retour au menu.
globals.c	saveNextIds()	Sauvegarde les compteurs next_client_id et next_account_id dans data/ids_counter.txt. Format: "next_client_id=[val]\nnext_account_id=[val]". Garantit continuité au redémarrage.
globals.c	loadNextIds()	Lit data/ids_counter.txt et initialise next_client_id et next_account_id. Appelé au démarrage pour reprendre la numérotation correcte depuis le dernier état sauvegardé.

3.1.2 Organisation des fichiers .c et .h:

tableau 8 : Organisation des fichiers .c et .h

Fichier	Type	Rôle	Responsabilités
main.c	Source	Point d'entrée	Initialise dossiers, charge IDs et données, lance l'application
menu.c	Source	Gestion Interface	Affiche tous les menus, gère la navigation et les interactions utilisateur
clients.c	Source	Gestion Clients	Opérations CRUD sur les clients (ajout, modification, suppression, recherche)
comptes.c	comptes.c	Gestion Comptes	Opérations CRUD sur les comptes bancaires (création, consultation, clôture)
operations.c	Source	Opérations Bancaires	Gère retraits et virements pour admin et clients avec validation
data.c	Source	Persistence Données	Charge/sauvegarde clients et comptes en/depuis fichiers CSV
utils.c	utils.c	Utilitaires	Validation, recherche, authentification, gestion buffer

globals.c	Source	Variables Globales	Définit et gère les variables partagées, compteurs ID, données en mémoire
donnees.h	Header	Structures Données	Définit structures Client et Compte, constantes (MAX, SOLDE_MIN, PLAFOND)
main.h	Header	Déclarations main	Prototypes et déclarations pour main.c
menu.h	Header	Déclarations menus	Prototypes de toutes les fonctions d'interface du menu
clients.h	Header	Déclarations clients	Prototypes des fonctions de gestion des clients
operations.h	Header	Déclarations comptes	Prototypes des fonctions de gestion des comptes
operations.h	Header	Déclarations opérations	Prototypes des fonctions de transactions bancaires
data.h	Header	Déclarations données	Déclarations données
utils.h	Header	Déclarations utilitaires	Prototypes des fonctions validation et recherche
globals.h	Header	Déclarations globales	Déclaration des variables globales et fonction gestion ID
CMakeLists.txt	Configuration	Build	Configure la compilation avec CMake (compilateur, sources, dossier output)

3.1.3 Focus sur les fonctionnalités clés (Extraits de code)

Plutôt que d'exposer l'intégralité du code source, qui s'étend sur plusieurs fichiers et centaines de lignes, nous avons choisi de présenter ici les fragments les plus significatifs du projet.

Ces extraits ont été sélectionnés car ils illustrent les défis techniques majeurs que nous avons relevés : la manipulation des pointeurs, la gestion des fichiers (lecture/écriture) et la sécurisation des saisies utilisateur. Ils montrent concrètement comment les algorithmes conçus au chapitre précédent ont été traduits en langage C.

figure13:Navigation avec Flèches (Menu Principal)

```
char demanderConfirmationAvecFleches() {
    char *options[] = {"Oui", "Non"};
    int selected = 0;
    int key;

    printf("\n%s", COLOR_INFO);
    printf(" > %s\n", options[0]);
    printf("%s    %s\n%s", COLOR_NORMAL, options[1], COLOR_RESET);

    while (1) {
        key = _getch();
        if (key == 224) { // Touche spéciale
            key = _getch();
            if (key == 72) { // Flèche UP
                selected = (selected - 1 + 2) % 2; // Wrapping
            } else if (key == 80) { // Flèche DOWN
                selected = (selected + 1) % 2; // Wrapping
            }
        } else if (key == 13) { // ENTREE
            return selected == 0 ? 'O' : 'N';
        }
        // Redessiner...
    }
}
```

La fonction `demanderConfirmationAvecFleches` remplace la saisie textuelle classique par une navigation interactive plus ergonomique. Elle utilise une boucle `while(1)` couplée à `_getch()` pour capturer les touches fléchées (codes 72 et 80) et déplacer un index de sélection de manière cyclique entre "Oui" et "Non". L'affichage est rafraîchi dynamiquement pour mettre en surbrillance l'option choisie par l'utilisateur. Enfin, la fonction valide le choix lors de l'appui sur la touche Entrée (code 13) et retourne le caractère correspondant ('O' ou 'N').

figure14:Validation avec Boucle Erreur/Rechargement)

```
void modifierClient() {
    while (1) {
        clearScreen();
        printClientHeader("MODIFICATION CLIENT");

        int clientId;
        printf("%s ID Client: %s", COLOR_NORMAL, COLOR_RESET);
        scanf("%d", &clientId);
        viderBuffer();

        int clientIdx = trouverIndexClient(clientId);
        if (clientIdx == -1) {
            clearScreen();
            printf("\n%s\n", COLOR_ERROR);
            printf("=====\n");
            printf(" [!] Client introuvable.\n");
            printf("=====\n%s\n", COLOR_RESET);
            Sleep(1500);
            clearScreen();
            continue; // Réessayer
        }

        // Modifier Les champs...

        saveClient(clients[clientIdx]);
        clearScreen();
        printf("%s\n Client modifié avec succès!\n%s\n", COLOR_SUCCESS, COLOR_RESET);
        Sleep(1000);
        clearScreen();
        break; // Quitter la boucle
    }
}
```

La fonction modifierClient sécurise la mise à jour des données en validant d'abord l'existence de l'ID client dans le tableau en mémoire via trouverIndexClient. Si l'identifiant est valide, les modifications sont immédiatement réécrites dans le fichier texte CSV via saveClient pour assurer la persistance des données. En cas d'erreur, une boucle de contrôle empêche l'accès à un profil inexistant et invite l'utilisateur à réessayer, garantissant ainsi la robustesse de l'application face aux saisies incorrectes.

figure15:Persistence des Transactions:

```
void saveWithdrawal(int clientId, int accountId, float amount, const char *date) {
    // Créer Le dossier data/retraits si nécessaire
    char folder[100];
    sprintf(folder, "data/retraits");
    _mkdir(folder);

    // Sauver dans fichier client
    char filename[150];
    sprintf(filename, "data/retraits/client_%d.txt", clientId);
    FILE *file = fopen(filename, "a");
    fprintf(file, "%d,%.2f,%s\n", accountId, amount, date);
    fclose(file);
}
```

La fonction `saveWithdrawal` garantit la traçabilité des flux financiers en ouvrant un fichier de log dédié au client en mode "ajout" ("a"), ce qui permet d'empiler les transactions sans écraser l'historique existant. Avant l'écriture, elle s'assure de l'existence du répertoire de stockage via `_mkdir` pour éviter tout plantage. Les détails du retrait (ID compte, montant, date) sont ensuite sérialisés au format CSV grâce à `fprintf`, assurant une lecture standardisée des données. Enfin, la fermeture explicite du fichier valide l'enregistrement persistant sur le disque.

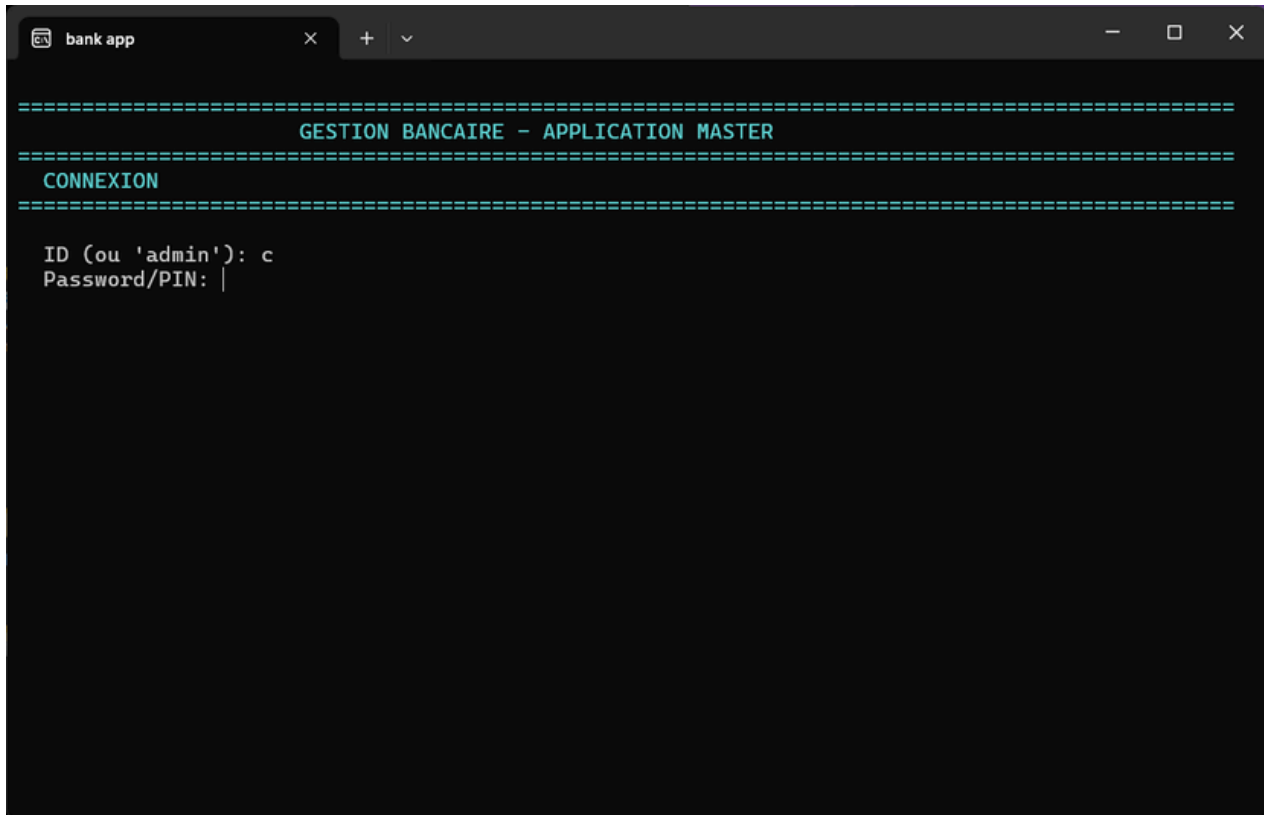
3.2 Environnement de Travail:

tableau 9:Environnement de Travail

Système d'exploitation	Windows 11		
Compilateur	GCC MinGW VS Code CMake	15.1.0 MSYS2 Dernière 3.31.6	Compilation C Suite complète compilateur IDE de développement Build optionnel
Langage	Langage C		
Bibliothèques utilisées	stdio.h stdlib.h string.h conio.h windows.h time.h	Input/Output Utilitaires Chaînes Console I/O API Windows Temps	Affichage console et fichiers Allocation mémoire, conversion Manipulation de strings Gestion clavier avancée (<code>getch()</code>) <code>Sleep()</code> , <code>_mkdir()</code> Date/heure des transactions

3.3 Résultats:

Figure 16 : Interface principale d'authentification et de connexion



3.3.1 admin:

Figure 17 : Menu Principal Administrateur avec navigation intuitive

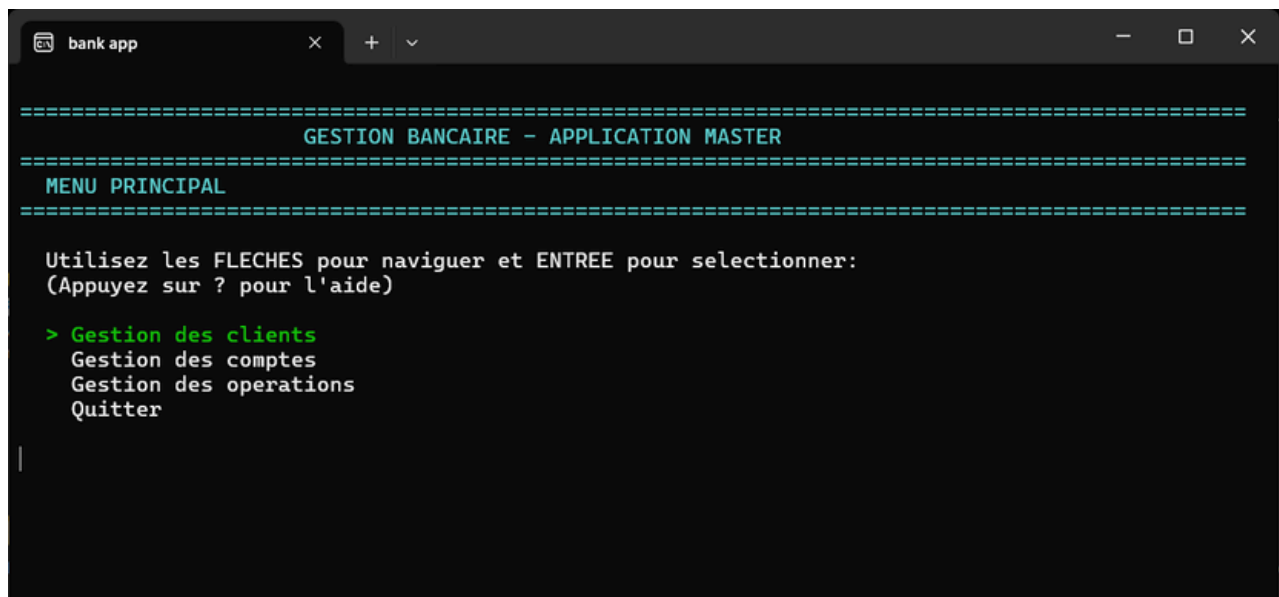


Figure 18 : Menu du sous-système de gestion des clients

```
bank app
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
GESTION DES CLIENTS
=====

Utilisez les FLECHES pour naviguer et ENTREE pour selectionner:

> Ajouter un client
  Modifier un client
  Supprimer un client
  Rechercher un client
  Afficher tous les clients
  Reinitialiser le PIN d'un client
  Retour au menu principal
```

Figure 19 : Formulaire interactif de création d'un nouveau client

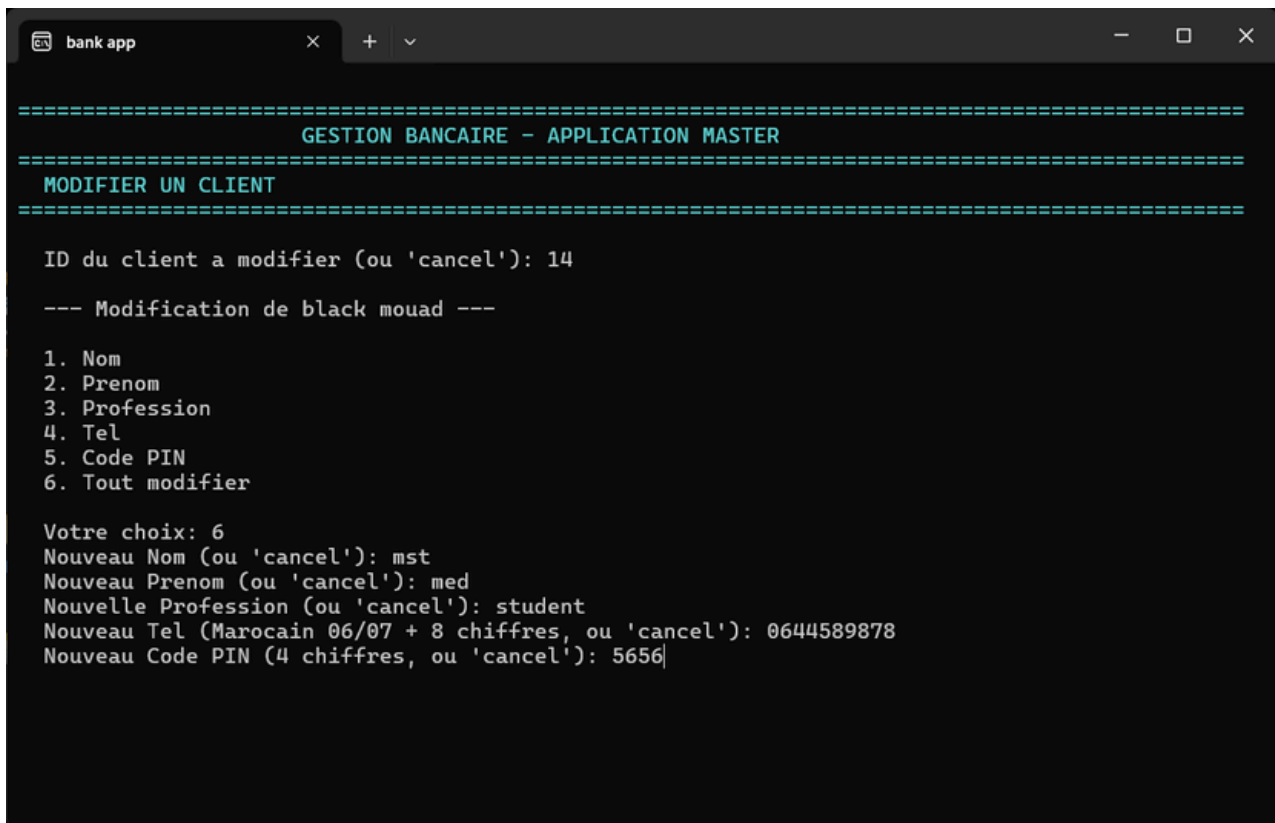
```
bank app
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
AJOUTER UN CLIENT
=====

ID Client (auto): 14

Nom (ou 'cancel'): black
Prenom (ou 'cancel'): mouad
Profession (ou 'cancel'): student
Num Tel (Marocain 06/07 + 8 chiffres, ou 'cancel'): 0620181444
Code PIN (4 chiffres, ou 'cancel'): 6767
```

Ce formulaire illustre le processus d'enrôlement où l'identifiant client est généré automatiquement (ici ID 14) pour garantir l'unicité absolue des dossiers. L'interface guide la saisie via des invites explicites qui imposent des règles de format strictes, notamment pour le numéro de téléphone et le code PIN. Cette validation interactive en amont assure l'intégrité des données avant leur persistance définitive dans le fichier CSV.

Figure 20 : Interface de modification granulaire des données client



```
bank app
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
MODIFIER UN CLIENT
=====

ID du client a modifier (ou 'cancel'): 14

--- Modification de black mouad ---

1. Nom
2. Prenom
3. Profession
4. Tel
5. Code PIN
6. Tout modifier

Votre choix: 6
Nouveau Nom (ou 'cancel'): mst
Nouveau Prenom (ou 'cancel'): med
Nouvelle Profession (ou 'cancel'): student
Nouveau Tel (Marocain 06/07 + 8 chiffres, ou 'cancel'): 0644589878
Nouveau Code PIN (4 chiffres, ou 'cancel'): 5656|
```

Cette fonctionnalité illustre la flexibilité du module de gestion, permettant à l'administrateur de mettre à jour soit un champ spécifique, soit l'intégralité du profil d'un client identifié par son ID. Le menu contextuel guide l'utilisateur vers l'option "Tout modifier", déclenchant une séquence de saisie sécurisée pour remplacer les anciennes informations (Nom, Tél, PIN). Une fois validées, ces nouvelles données écrasent immédiatement les précédentes dans le système de persistance CSV, assurant ainsi la mise à jour en temps réel.

Figure 21 : Procédure sécurisée de suppression d'un client

```
bank app  x + v

=====
GESTION BANCAIRE - APPLICATION MASTER
=====
SUPPRIMER UN CLIENT
=====

ID Client a supprimer (ou 'cancel'): 14|

bank app  x + v

=====
GESTION BANCAIRE - APPLICATION MASTER
=====
CONFIRMATION SUPPRESSION
=====

Informations du client:
- ID: 14
- Nom: mst
- Prenom: med
- Profession: student
- Tel: 0644589878

Etes-vous sur de vouloir supprimer ce client?

> Oui
Non
|
```

La procédure de suppression intègre un mécanisme de sécurité à double détente : elle requiert d'abord l'identification du client cible via son ID unique, puis affiche une fiche récapitulative complète pour une validation visuelle avant destruction. L'action irréversible est conditionnée par une confirmation explicite "Oui/Non", agissant comme un garde-fou indispensable contre les effacements accidentels de données sensibles ou financières.

Figure 22 : Module de recherche multicritère de client Explication

```

=====
GESTION BANCAIRE - APPLICATION MASTER
=====
RECHERCHER UN CLIENT
=====

Rechercher par: 1. ID 2. Nom (ou 'cancel'): 1
ID (ou 'cancel'): 14

-> ID: 14 | mst med | Job: student | Tel: 0644589878
  
```

Cette fonctionnalité optimise l'accès à l'information en offrant une recherche flexible par identifiant unique ou par nom de famille. Comme illustré, la saisie de l'ID "14" déclenche une requête instantanée qui restitue le profil complet du client (Identité, Profession, Téléphone) en surbrillance verte. Ce mécanisme permet à l'administrateur de vérifier rapidement l'existence et les détails d'un compte sans avoir à parcourir manuellement la liste complète des utilisateurs.

Figure 23 : Affichage tabulaire de la liste complète des clients

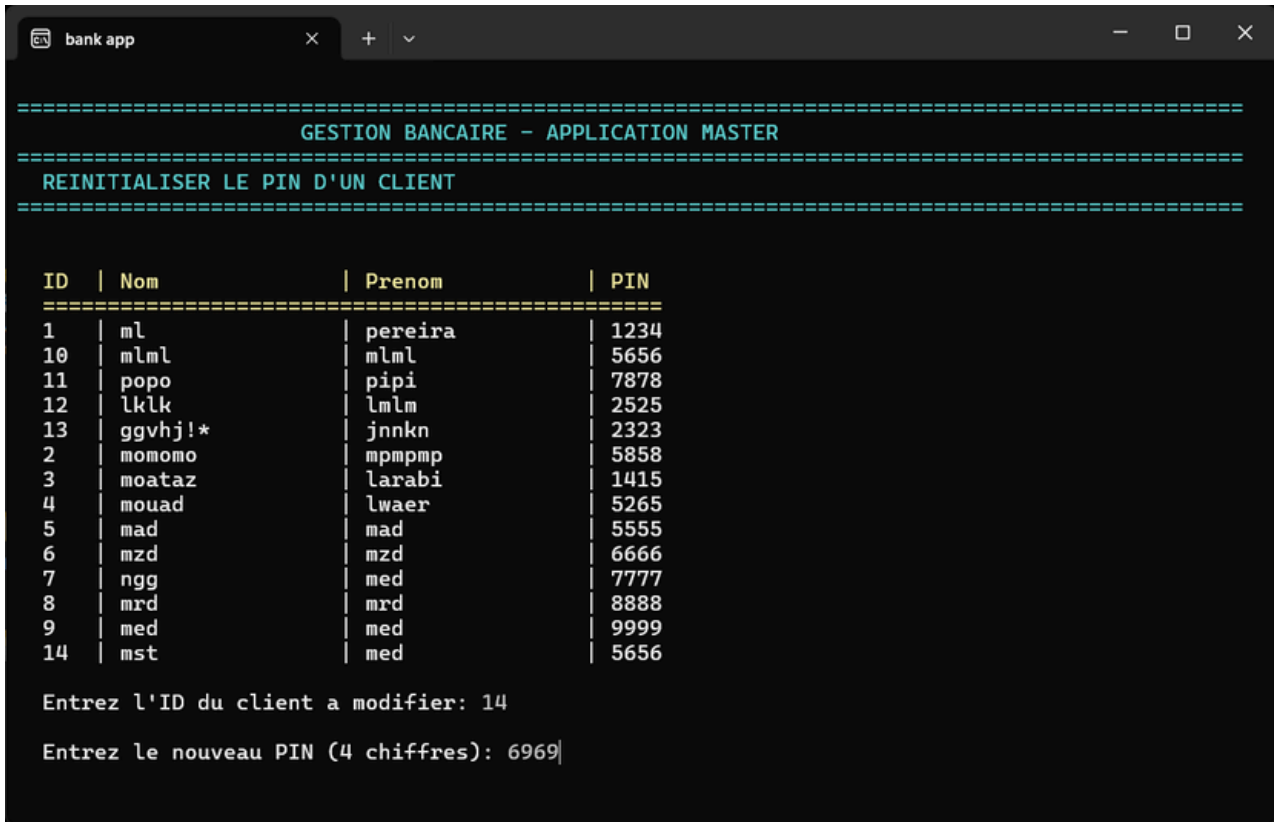
```

=====
GESTION BANCAIRE - APPLICATION MASTER
=====
LISTE DE TOUS LES CLIENTS
=====

ID | Nom | Prenom | Profession | Telephone | PIN
=====
1 | ml | pereira | f | 0620305060 | 1234
2 | momomo | mpmpmp | std | 0656565656 | 5858
3 | moataz | larabi | fp | 0705649584 | 1415
4 | mouad | lwaer | g | 0687958465 | 5265
5 | mad | mad | hjjjjjjjj | 0655555555 | 5555
6 | mzd | mzd | mzd | 0666666666 | 6666
7 | ngg | med | med | 0777777777 | 7777
8 | mrd | mrd | mrd | 0788888888 | 8888
9 | med | med | khbg | 0677777777 | 9999
10 | mlml | mlml | hgth | 0625582525 | 5656
11 | popo | pipi | s | 0723232323 | 7878
12 | lklk | lmlm | s | 0666633662 | 2525
13 | ggvhj!* | jnnkn | s | 0666666252 | 2323
14 | mst | med | student | 0644589878 | 5656

Total: 14 client(s) enregistre(s)
  
```

Figure 24 : Interface de réinitialisation administrative du code PIN



```

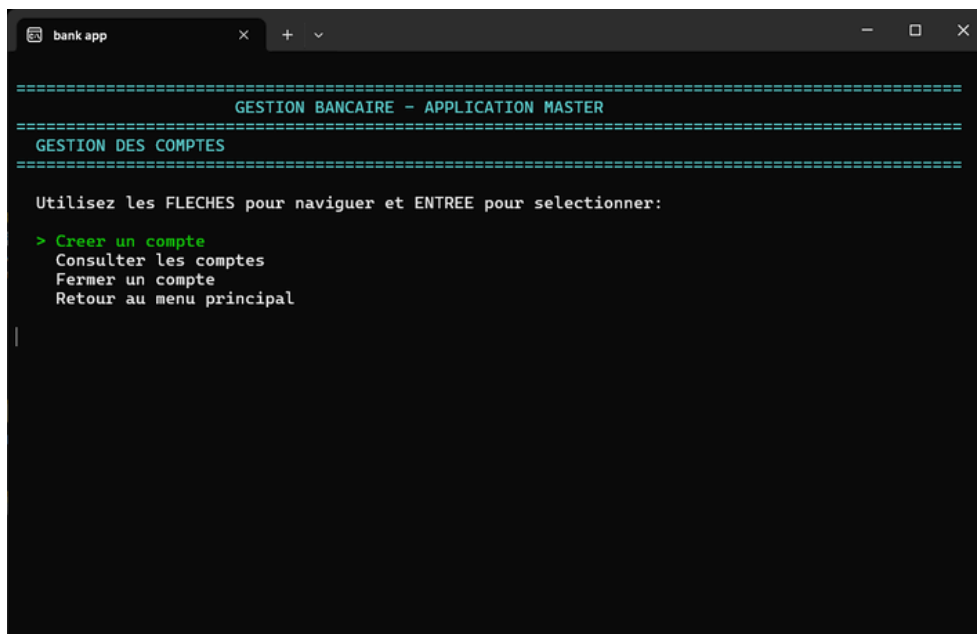
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
REINITIALISER LE PIN D'UN CLIENT
=====

ID | Nom          | Prenom      | PIN
=====
1  | ml           | pereira     | 1234
10 | mlml        | mlml        | 5656
11 | popo        | pipi        | 7878
12 | lklk        | lmlm        | 2525
13 | ggvhj!*     | jnnkn       | 2323
2  | momomo      | mpmpmp     | 5858
3  | moataz      | larabi      | 1415
4  | mouad       | lwaer       | 5265
5  | mad         | mad         | 5555
6  | mzd         | mzd         | 6666
7  | ngg         | med         | 7777
8  | mrd         | mrd         | 8888
9  | med         | med         | 9999
14 | mst         | med         | 5656

Entrez l'ID du client a modifier: 14
Entrez le nouveau PIN (4 chiffres): 6969
  
```

Cette fonctionnalité de maintenance permet à l'administrateur de restaurer l'accès d'un utilisateur en cas d'oubli de ses identifiants. L'interface affiche d'abord une liste de référence pour faciliter l'identification du compte cible (ici l'ID 14) avant de solliciter la saisie du nouveau code confidentiel. Cette opération sensible met à jour directement le profil sécurisé du client, rendant le nouveau PIN immédiatement effectif pour les futures connexions

Figure 25 : Menu centralisé de gestion des comptes bancaires



```

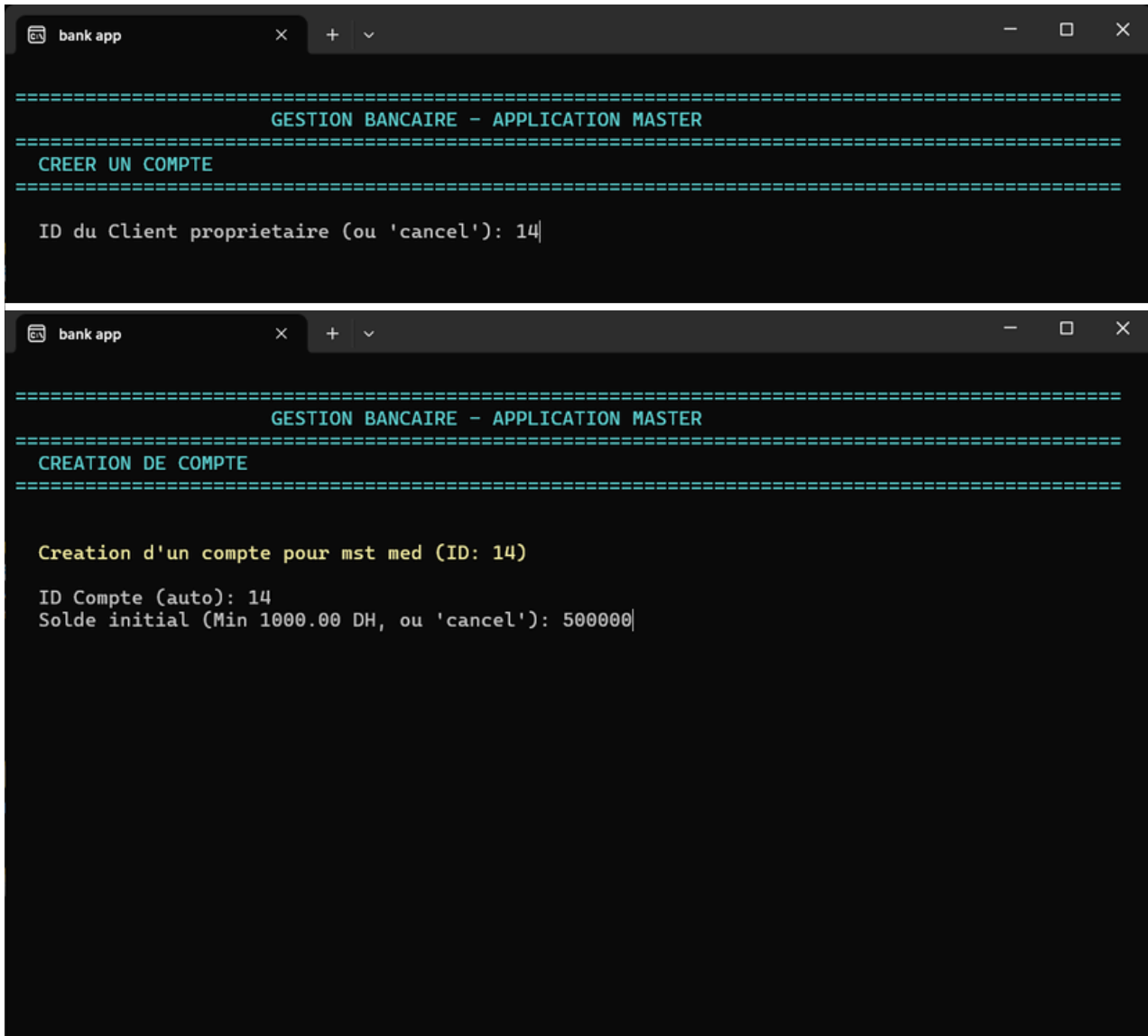
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
GESTION DES COMPTES
=====

Utilisez les FLECHES pour naviguer et ENTREE pour selectionner:

> Créer un compte
  Consulter les comptes
  Fermer un compte
  Retour au menu principal

|
  
```

Figure 26 : Processus de création et provisionnement d'un compte



The image shows two screenshots of the 'bank app' interface. The top screenshot displays the 'GESTION BANCAIRE - APPLICATION MASTER' menu with the option 'CREER UN COMPTE'. The user has entered '14' for the client ID. The bottom screenshot shows the 'CREATION DE COMPTE' screen, indicating the account is being created for 'mst med (ID: 14)'. It shows the 'ID Compte (auto): 14' and the 'Solde initial (Min 1000.00 DH, ou 'cancel'): 500000'.

```
bank app
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
CREER UN COMPTE
=====

ID du Client propriétaire (ou 'cancel'): 14|

bank app
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
CREATION DE COMPTE
=====

Creation d'un compte pour mst med (ID: 14)

ID Compte (auto): 14
Solde initial (Min 1000.00 DH, ou 'cancel'): 500000|
```

L'interface de création lie automatiquement un nouveau compte (ID 14) au client sélectionné, tout en appliquant une règle métier stricte sur le solde initial (minimum 1000 DH). Cette étape de validation financière est un prérequis obligatoire pour garantir la viabilité du compte avant son enregistrement. Une fois le montant validé, le système génère instantanément le fichier unique du compte dans le répertoire du client, assurant une parfaite intégrité relationnelle des données.

Figure 27 : Vue consolidée du parc de comptes bancaires Explication

bank app

```
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
LISTE DES COMPTES
=====
```

ID Compte	ID Client	Solde (DH)	Date Ouverture
11	1	84984.00	29/01/2026
3	1	107629.00	28/01/2026
10	11	1000.00	29/01/2026
7	11	5668041.00	29/01/2026
9	11	654165.00	29/01/2026
2	2	260164.00	28/01/2026
8	4	5618548736.00	29/01/2026
6	5	666666.00	28/01/2026
13	14	54545.00	30/01/2026
14	14	500000.00	30/01/2026

Figure 28 : Procédure sécurisée de clôture d'un compte bancaire

bank app

```
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
FERMER UN COMPTE
=====

ID Compte a fermer (ou 'cancel'): 14
```

bank app

```
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
CONFIRMATION FERMETURE
=====

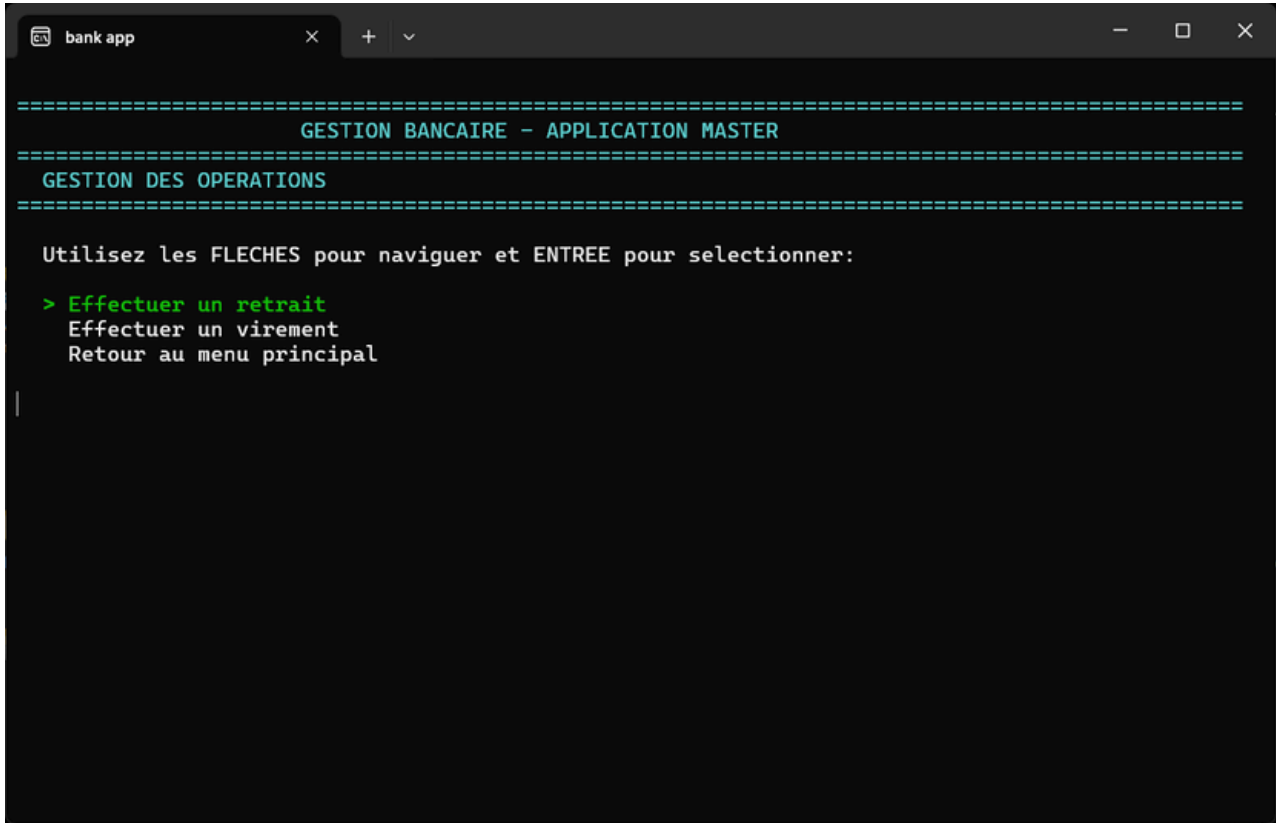
Informations du compte:
- ID Compte: 14
- ID Client: 14
- Solde: 500000.00 DH
- Date ouverture: 30/01/2026

Etes-vous sur de vouloir fermer ce compte?

> Oui
  Non
```

Cette fonctionnalité encadre la suppression d'un compte en imposant une vérification rigoureuse pour éviter toute erreur irréversible sur les actifs financiers. Après identification du compte cible (ID 14), le système génère un rapport de synthèse affichant le solde final et la date d'ouverture pour validation visuelle. La clôture définitive est alors conditionnée par une confirmation explicite via un sélecteur interactif, agissant comme un ultime garde-fou avant la destruction du fichier.

Figure 29 : Menu de sélection des opérations financières

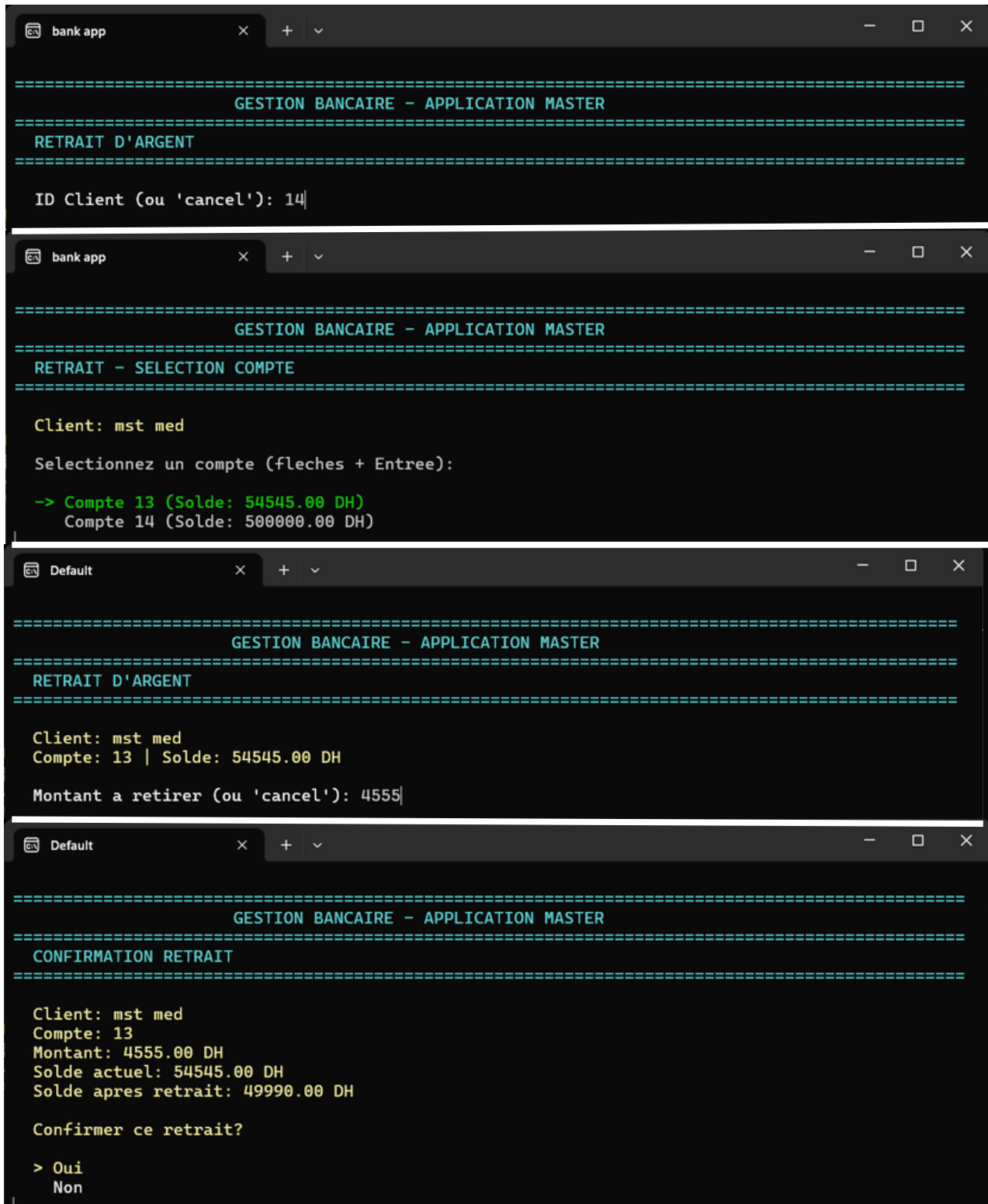


```
bank app
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
GESTION DES OPERATIONS
=====

Utilisez les FLECHES pour naviguer et ENTREE pour selectionner:

> Effectuer un retrait
  Effectuer un virement
  Retour au menu principal
|
```

Figure 30 : Séquence interactive et sécurisée de retrait d'espèces

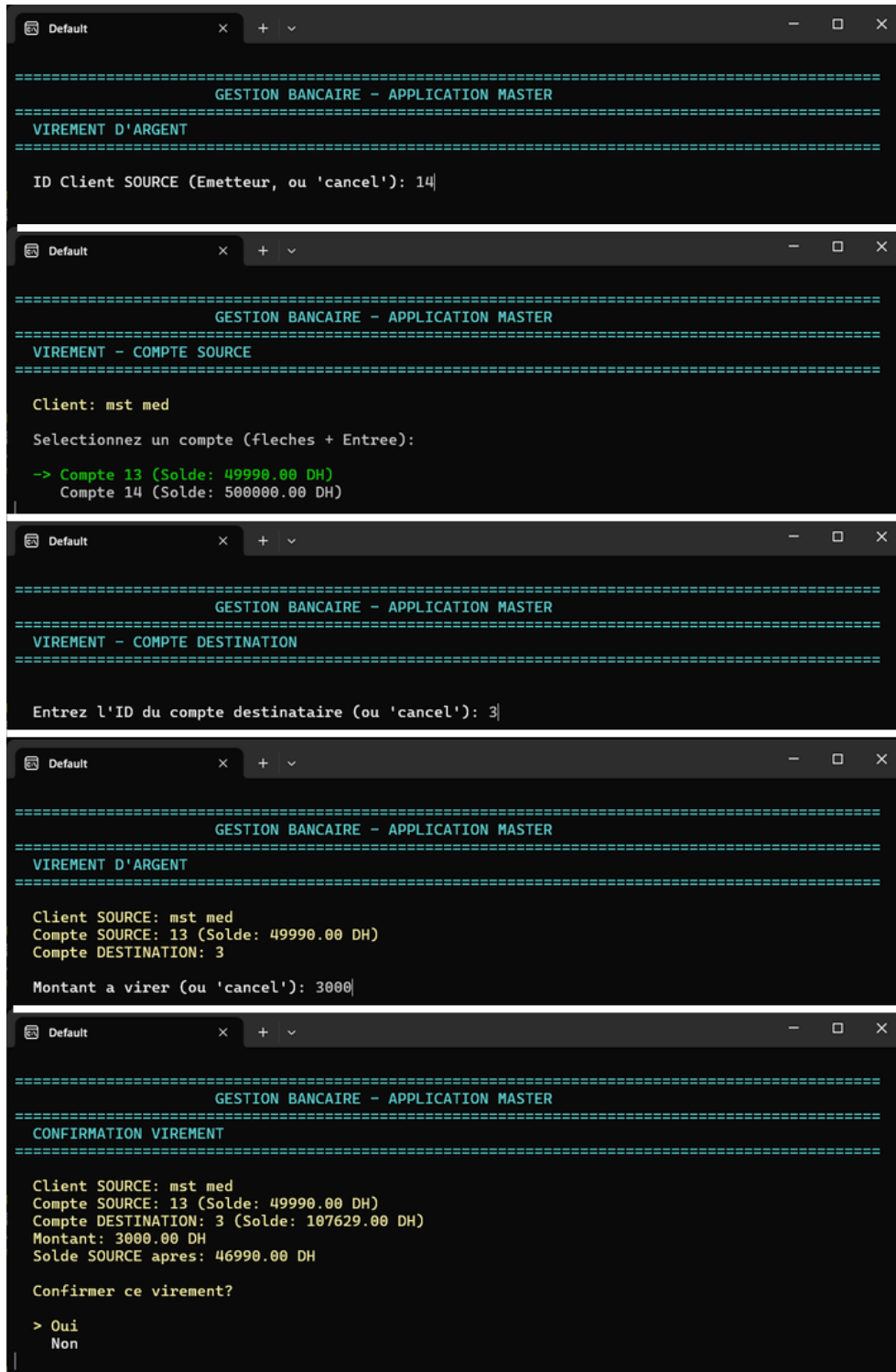


The figure displays four sequential screenshots of the cBank App interface, illustrating the cash withdrawal process:

- Screen 1:** The application title is "GESTION BANCAIRE - APPLICATION MASTER". The menu option "RETRAIT D'ARGENT" is selected. The user enters the client ID "14".
- Screen 2:** The application title remains "GESTION BANCAIRE - APPLICATION MASTER". The menu option "RETRAIT - SELECTION COMPTE" is selected. The user is prompted to "Selectionnez un compte (fleches + Entree):". The screen displays two account options: "Compte 13 (Solde: 54545.00 DH)" and "Compte 14 (Solde: 500000.00 DH)".
- Screen 3:** The application title is "GESTION BANCAIRE - APPLICATION MASTER". The menu option "RETRAIT D'ARGENT" is selected. The user is prompted to enter the "Montant a retirer (ou 'cancel')". The screen displays the client information: "Client: mst med" and "Compte: 13 | Solde: 54545.00 DH".
- Screen 4:** The application title is "GESTION BANCAIRE - APPLICATION MASTER". The menu option "CONFIRMATION RETRAIT" is selected. The screen displays the client information: "Client: mst med", "Compte: 13", "Montant: 4555.00 DH", "Solde actuel: 54545.00 DH", and "Solde apres retrait: 49990.00 DH". The user is prompted to "Confirmer ce retrait?" with options "> Oui" and "Non".

Ce flux transactionnel guide l'opérateur étape par étape, débutant par la sélection intuitive du compte à débiter parmi les avoirs du client suivie de la saisie du montant. Le système sécurise l'opération par un écran de simulation affichant le solde avant et après retrait, permettant de vérifier la provision en temps réel. La transaction n'est actée qu'après une confirmation explicite, garantissant l'intégrité comptable et prévenant les erreurs de manipulation.

Figure 31 : Workflow sécurisé de virement inter-comptes



```

Default x + v
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
VIREMENT D'ARGENT
=====
ID Client SOURCE (Emetteur, ou 'cancel'): 14|

Default x + v
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
VIREMENT - COMPTE SOURCE
=====
Client: mst med
Selectionnez un compte (fleches + Entree):
-> Compte 13 (Solde: 49990.00 DH)
    Compte 14 (Solde: 500000.00 DH)

Default x + v
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
VIREMENT - COMPTE DESTINATION
=====
Entrez l'ID du compte destinataire (ou 'cancel'): 3|

Default x + v
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
VIREMENT D'ARGENT
=====
Client SOURCE: mst med
Compte SOURCE: 13 (Solde: 49990.00 DH)
Compte DESTINATION: 3
Montant a virer (ou 'cancel'): 3000|

Default x + v
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
CONFIRMATION VIREMENT
=====
Client SOURCE: mst med
Compte SOURCE: 13 (Solde: 49990.00 DH)
Compte DESTINATION: 3 (Solde: 107629.00 DH)
Montant: 3000.00 DH
Solde SOURCE apres: 46990.00 DH
Confirmer ce virement?
> Oui
  Non
  
```

Le module de virement structure l'échange de fonds via une séquence rigoureuse : sélection du compte émetteur, identification du destinataire et saisie du montant. Un écran de récapitulatif final verrouille la transaction en affichant les impacts sur les soldes avant toute validation définitive. Cette procédure à étapes multiples garantit l'exactitude des données bancaires et prévient les erreurs de routage financier avant l'exécution irréversible.

3.3.2 client:

Figure 32 : Tableau de bord de l'espace client

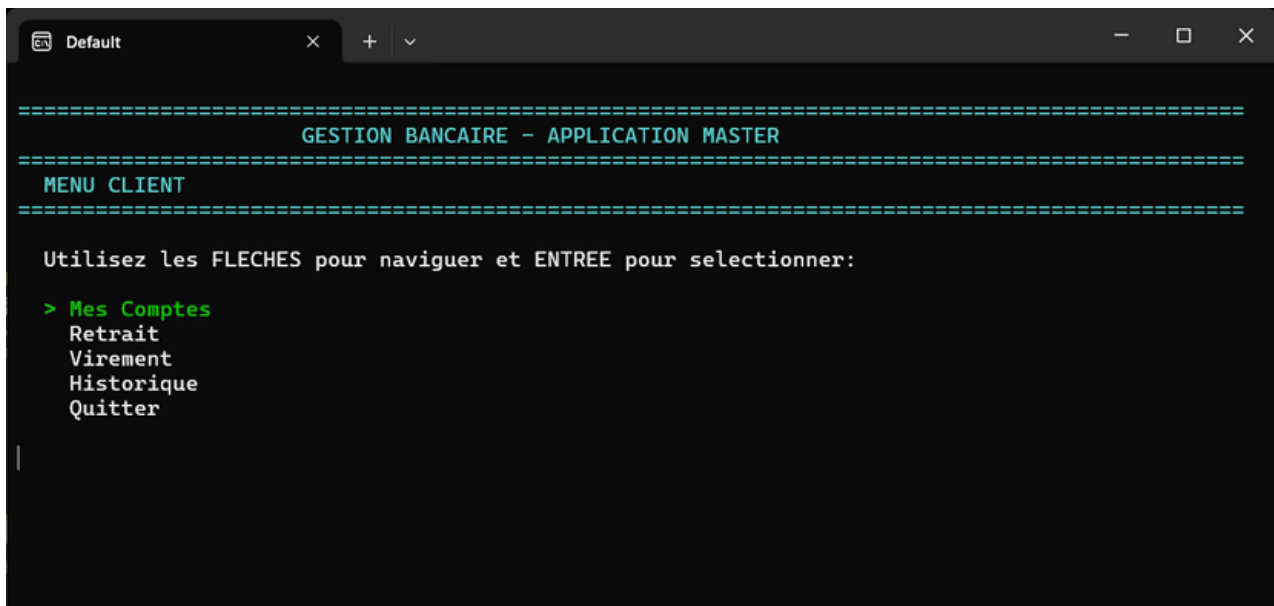


Figure 33 : Interface de consultation du portefeuille personnel

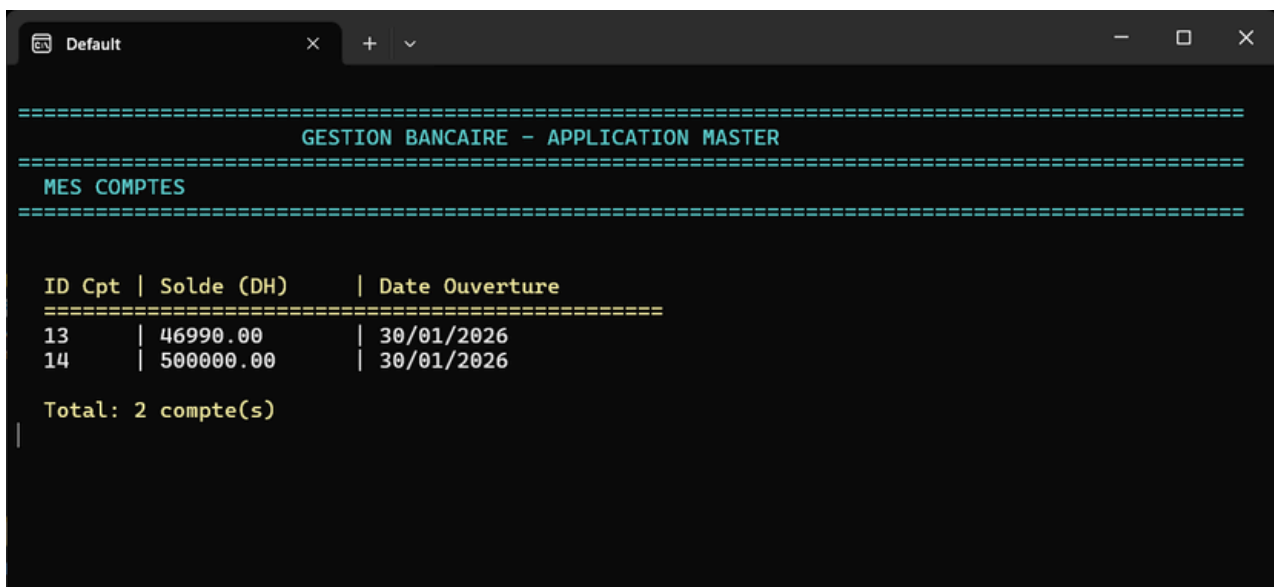
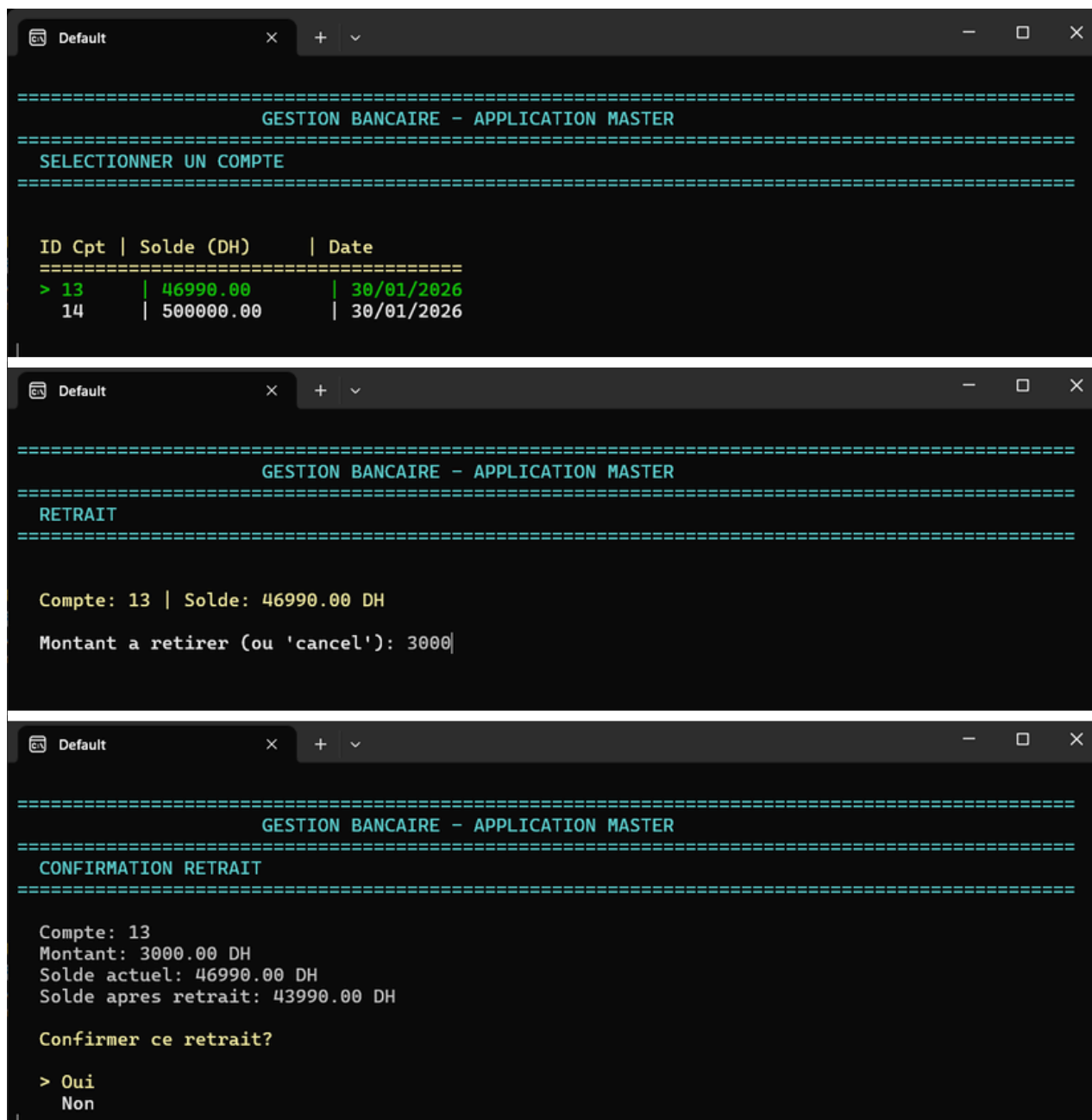


Figure 34 : Processus de retrait d'espèces depuis l'espace client



GESTION BANCAIRE - APPLICATION MASTER

SELECTIONNER UN COMPTE

ID Cpt	Solde (DH)	Date
> 13	46990.00	30/01/2026
14	500000.00	30/01/2026

GESTION BANCAIRE - APPLICATION MASTER

RETRAIT

Compte: 13 | Solde: 46990.00 DH

Montant a retirer (ou 'cancel'): 3000|

GESTION BANCAIRE - APPLICATION MASTER

CONFIRMATION RETRAIT

Compte: 13
Montant: 3000.00 DH
Solde actuel: 46990.00 DH
Solde apres retrait: 43990.00 DH

Confirmer ce retrait?

> Oui
Non

Cette fonctionnalité offre une autonomie complète au client pour ses retraits, débutant par la sélection du compte source via une interface simplifiée. Après la saisie du montant, le système sécurise l'acte par une simulation financière montrant l'impact immédiat sur le solde. Cette validation à double niveau (saisie puis confirmation) prévient les erreurs de manipulation et garantit une gestion transparente de la trésorerie personnelle.

Figure 35 : Exécution sécurisée d'un virement bancaire par le client



```

=====
GESTION BANCAIRE - APPLICATION MASTER
=====
SELECTIONNER UN COMPTE
=====

ID Cpt | Solde (DH) | Date
=====
> 13 | 43990.00 | 30/01/2026
  14 | 500000.00 | 30/01/2026

=====
GESTION BANCAIRE - APPLICATION MASTER
=====
VIREMENT - COMPTE DESTINATION
=====

Entrez l'ID du compte destinataire (ou 'cancel'): 3|

=====
GESTION BANCAIRE - APPLICATION MASTER
=====
VIREMENT - COMPTE DESTINATION
=====

Entrez l'ID du compte destinataire (ou 'cancel'): 3
Compte SOURCE: 13 (Solde: 43990.00 DH)
Compte DESTINATION: 3
Montant a virer (ou 'cancel'): 3000|

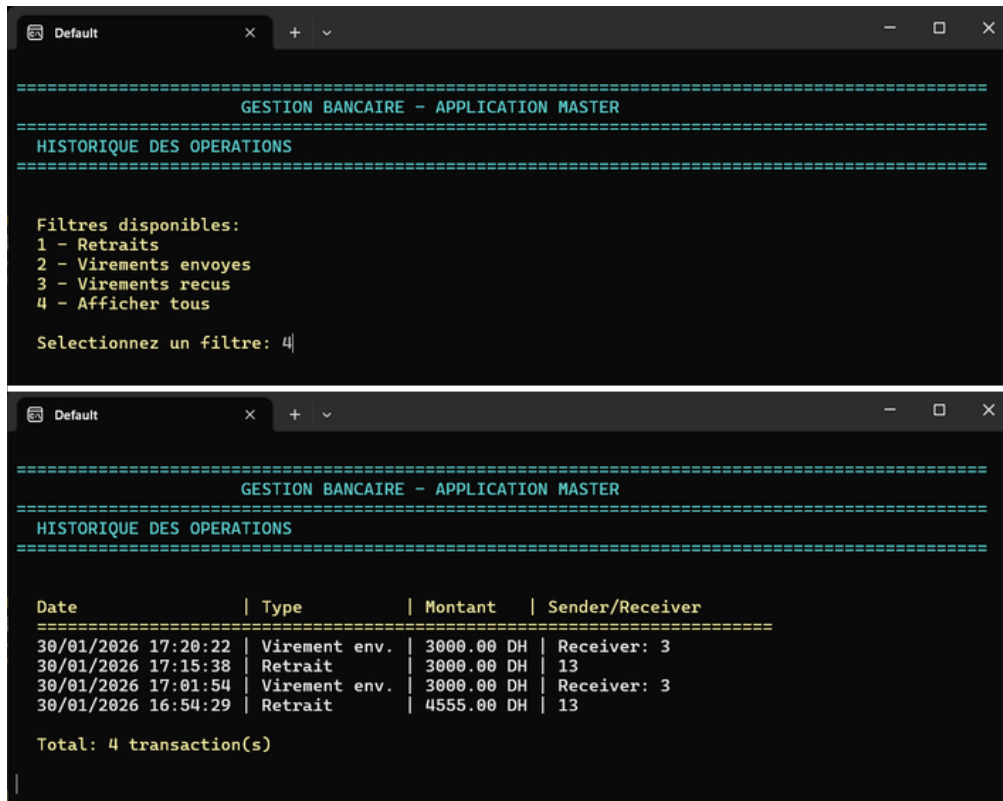
=====
GESTION BANCAIRE - APPLICATION MASTER
=====
CONFIRMATION VIREMENT
=====

Compte SOURCE: 13 (Solde: 43990.00 DH)
Compte DESTINATION: 3
Montant: 3000.00 DH
Solde SOURCE apres: 40990.00 DH

Confirmer ce virement?
> Oui
  Non
  
```

Cette fonctionnalité permet au client d'effectuer des virements en toute autonomie, débutant par le choix du compte émetteur et l'identification du bénéficiaire. La procédure se poursuit par la saisie du montant, avec un contrôle immédiat de la disponibilité des fonds affiché à l'écran. Un récapitulatif détaillé incluant le solde prévisionnel conclut le processus, exigeant une confirmation explicite pour valider la transaction et garantir la sécurité des échanges inter-comptes.

Figure 36 : Module de consultation et de filtrage de l'historique des transactions



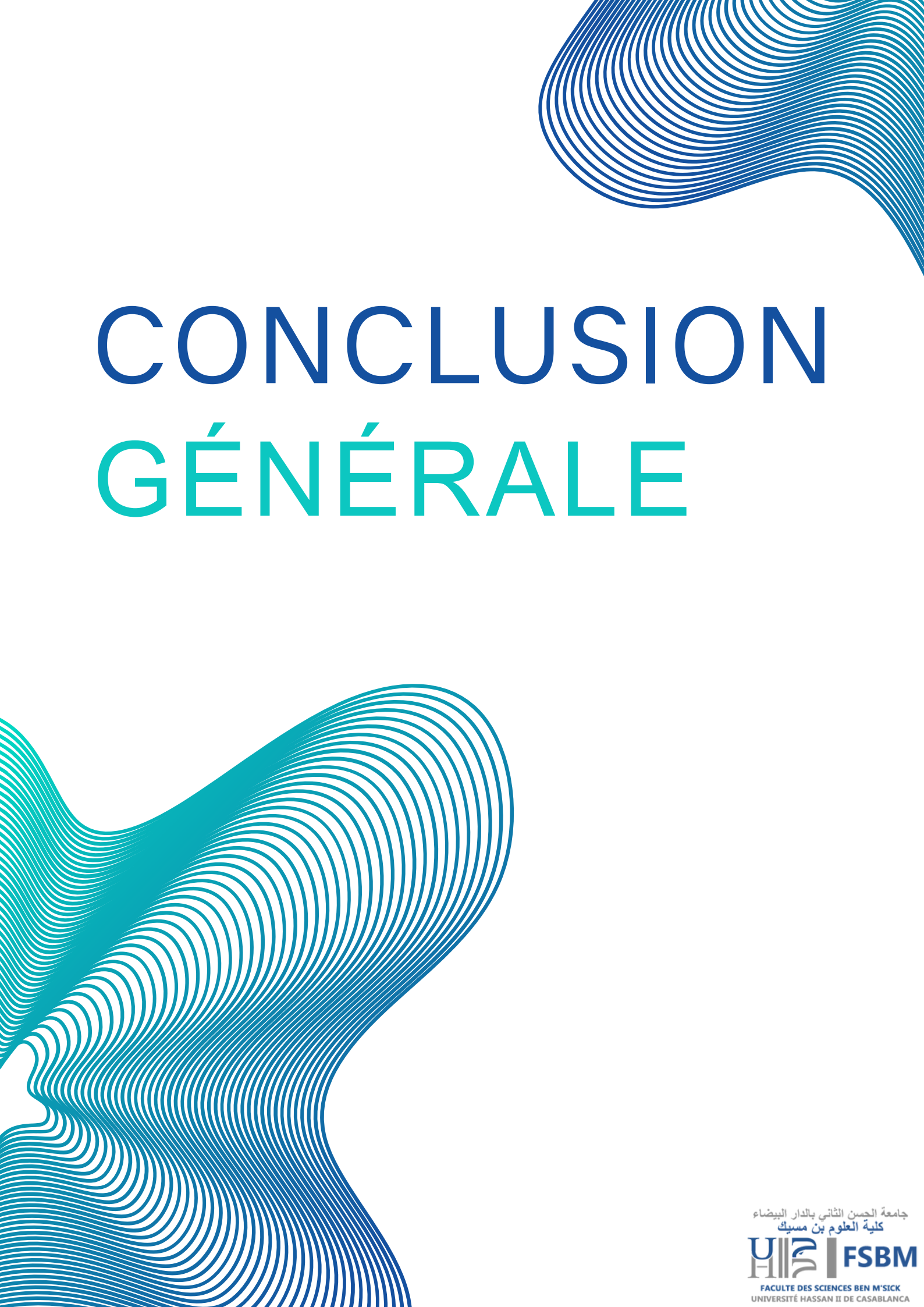
Ce module assure une traçabilité complète des activités financières, permettant au client de filtrer ses opérations (retraits, virements émis/reçus) pour affiner sa recherche. Les résultats s'affichent sous forme de journal chronologique détaillé, précisant pour chaque ligne la date, le type de transaction, le montant et le tiers concerné. Cette fonctionnalité d'audit permet à l'utilisateur de vérifier la conformité de ses comptes en offrant une transparence totale sur les mouvements de fonds passés. La présentation tabulaire structurée facilite la lecture rapide et le suivi rigoureux de l'évolution du solde.

conclusion:

conclusion, ce chapitre a permis d'illustrer la concrétisation fonctionnelle et technique de notre application de gestion bancaire. À travers le parcours complet des interfaces, nous avons démontré la capacité du système à couvrir l'intégralité des besoins identifiés : de l'administration rigoureuse des profils clients et bancaires à la gestion sécurisée des transactions financières (retraits et virements).

L'approche choisie, combinant une navigation ergonomique au clavier et une persistance des données fiable via le format CSV, garantit à la fois la fluidité de l'expérience utilisateur et la traçabilité des opérations. La séparation stricte entre les espaces "Administrateur" et "Client", ainsi que l'intégration de modules d'audit comme l'historique filtrable, attestent de la robustesse et de la maturité du prototype développé.

CONCLUSION GÉNÉRALE



CONCLUSION GÉNÉRALE

جامعة الحسن الثاني بالدار البيضاء
كلية العلوم بن مسبك

UHA | **FSBM**

FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

Bilan du Projet

Ce projet de système de gestion bancaire a abouti au développement d'une application fonctionnelle écrite en langage C (ISO C99), compilable avec GCC 15.1.0 sur plateforme Windows x86-64. L'application propose une interface complète de gestion de comptes clients, transactions bancaires, et persistance des données via le système de fichiers.

Points Clés du Bilan :

Architecture modulaire : 8 modules (main, menu, clients, comptes, operations, data, utils, globals) avec séparation claire des responsabilités

Couverture fonctionnelle : 62 fonctions implémentées couvrant authentification, CRUD clients/comptes, transactions, validation, et persistance

Documentation exhaustive : 1500+ lignes de documentation technique avec exemples réels du code source

Compilation réussie : Absence d'erreurs de compilation, exécutable fonctionnel généré

Stockage hiérarchisé : Structure de fichiers organisée par client avec séparation logique des comptes et opérations

Objectifs Atteints ou Non

tableau 10: Objectifs Atteints ou Non

Objectifs Atteints	Authentification admin/clients, CRUD clients/comptes, opérations bancaires (retrait/virement), validation données robuste, persistance fichier hiérarchisée, historique transactions, génération IDs uniques
Objectifs Non Atteints	Interface graphique (console texte uniquement), sécurité avancée (pas de chiffrement/hash), base de données (fichiers texte), portabilité cross-platform (Windows-specific), reporting statistiques, gestion concurrence utilisateurs.

Difficultés Rencontrées

- **Gestion mémoire** : Allocations dynamiques sans outils automatiques, risque fuites
- **Persistance fichier** : Intégrité limitée en corruption, lenteur données massives
- **Validation** : Buffer overflow potentiel, formats invalides
- **Dépendances Windows** : Code non-portable (conio.h, direct.h, windows.h)
- **Synchronisation** : Pas de verrouillage accès concurrent

Améliorations Possibles

tableau 11:Améliorations Possibles

Interface Graphique → GTK 4	- Menu navigable avec curseur - Fenêtres dialogues pour saisies - Affichage tabulaire données - Icônes et thématisation moderne - Cross-platform (Windows/Linux/macOS)
Sécurité	- Hash mots de passe (bcrypt/SHA-256) - Chiffrement données sensibles (AES) - Contrôle accès granulaire (RBAC)
Base de Données	- SQLite (léger, intégré) ou PostgreSQL - Transactions ACID - Indexation et performances
Fonctionnalités	- Intérêts composés automatiques - Limites retrait quotidien - Alertes solde faible - Export CSV/PDF

Ouvertures Futures

Le projet peut évoluer vers une plateforme bancaire digitale complète : court terme (3-6 mois) avec microservices REST et interface web moderne (React/Vue.js) ; moyen terme (6-12 mois) avec investissements, multi-devises, paiements mobiles et authentification biométrique ; long terme (12+ mois) avec IA fraude/recommandations, blockchain/crypto, et expansion multi-agence. Vision ultime : néobanque full-stack avec licence officielle, support assurance/retraite/courtage, et écosystème financier innovant couvrant fintech, DeFi, et métaverse banking — à condition de sécuriser d'abord (chiffrement, hash, 2FA), basculer sur BD robuste, puis moderniser l'interface.



cBank App

“GESTION DES CLIENTS ET DES COMPTES BANCAIRES” **PROJET DE FIN DE MODULE EN LANGAGE C** Département de Mathématiques et Informatique

“Ce projet consiste en la réalisation d'une application de gestion bancaire complète en langage C. Elle permet l'administration sécurisée des clients, la gestion des comptes et l'exécution de transactions financières (virements, retraits) avec une sauvegarde automatique des données sur fichiers. Ce prototype modulaire valide la maîtrise de la programmation structurée et de l'algorithmique complexe.

Merci de votre lecture.

Ce projet marquant une étape dans mon apprentissage, vos retours et suggestions sont les bienvenus.”

Mohammed MOUSTAFID